

UNIVERSITY OF KENT

MA 876

PROJECT

N-Mixture Models from a Bayesian Perspective

Author

████████████████████

Supervisor

████████████████████

Student number: ██████████

Submitted in partial fulfilment of the requirements of the degree of MSc Statistics,
University of Kent, 2018-2019

August 30, 2019

Acknowledgements

I would like to thank my supervisor [REDACTED] and [REDACTED] for their consistent support and guidance during the running of this project. [REDACTED] continuously provided encouragement [REDACTED]
[REDACTED] for supporting me during the compilation of this dissertation.

Abstract

This project was motivated by count data on yellow bellied toads (*Bombina variegata*) and the need to obtain an estimate of absolute abundance of the population. Using only replicated counts from unmarked individuals, N-mixture models provide an attractive framework to gain inference on absolute abundance rather than relative abundance. Consequently, new N-mixture models introduced by the researcher as well as existing N-mixture models were fitted to the count data on yellow bellied toads to obtain an estimate of absolute abundance. It was found that the different N-mixture models produced hugely different estimates of absolute abundance. The Wantanabe-Akaike information criterion (WAIC) was used to select the N-mixture model that fitted the data best. Subsequently, simulations were performed to assess whether WAIC can be used to discriminate between the different N-mixture models considered in this project. The results obtained suggest that WAIC is not a reliable tool for discriminating between N-mixture models as it only identified the true model in some scenarios and failed to identify the true model in others. Notably, it was also found that N-mixture models with Beta-Binomial detection processes can be supported by WAIC even when the underlying truth is that there exists over-dispersion or equi-dispersion in abundance process. These N-mixture models with Beta-Binomial detection processes also tend to underestimate absolute abundance in these case. Due to these results, model selection of N-mixture models via WAIC should be treated with caution.

Motivated by identifiability issues in N-mixture models, this project addresses a number of identifiability issues in a Bayesian framework. This study via simulations investigated: whether one can identify detection probability of individuals from count data by numerically computing the overlap between pairs of prior and posterior distributions, whether one can unambiguously discriminate between different hierarchical representations of count data by following the Barker et al. (2018) simulation settings and whether one can unequivocally discriminate between N-mixture models by using the Savage Dickey Density ratio (SDDR). Results obtained from numerically computing the overlap between pairs of prior and posterior distributions showed that in most cases one can identify detection probability from count data using N-mixture models with Binomial detection processes. Hence, N-mixture models with Binomial detection processes can provide valuable information on detection probability and hence population size. Based on WAIC, it seems that under certain scenarios one cannot unequivocally discriminate between different hierarchical representations of count data, some of which do not hold any inference on abundance. However, results obtained from applying the SDDR suggest that under certain scenarios, especially when detection probability is large, one can unequivocally discriminate between N-mixture models. Hence, in this project, cases where one can reliably discriminate between different N-mixture models and also estimate abundance separately from detection were found.

Contents

1	Introduction	4
2	Mathematical Background	5
2.1	Poisson distribution	5
2.2	Discrete Weibull (DW) Distribution	5
2.3	Negative Binomial (NB) distribution	6
2.4	Double Poisson (DP) distribution	6
2.5	Zero inflated Poisson (ZIP) distribution	7
3	Models	7
3.1	N-mixture Models	7
3.2	Why use Bayesian inference on N-mixture Models?	9
3.3	Software	9
3.3.1	Creating user-defined BUGS distribution in NIMBLE	10
3.3.2	Fitting N-mixture models	10
4	Model Checking	11
4.1	Posterior Predictive check	11
4.2	Predictive Accuracy	11
4.2.1	Wantanable-Akaike information criterion (WAIC)	12
4.3	Bias of Absolute Abundance Estimates	13
4.4	MCMC diagnostics	13
4.5	Model Identifiability	13
4.5.1	Practical Computation of τ	14
4.6	Savage Dickey density ratio (SDDR)	15
4.6.1	Computation of the Savage Dickey density ratio	15
5	Analysis of yellow bellied toads count data	16
6	Simulation	18
6.1	Identifiability of N-mixture models based on WAIC along with test for de- tection probability identifiability	18
6.2	Can one discriminate among different hierarchical representations of count data?	21
6.3	Identifiability of N-mixture models using Savage Dickey Density ratio	23
7	Discussion	25
	References	27
	Appendices	29
A	Fitting N-mixture models in NIMBLE	29

1 Introduction

Estimating abundance of a population is a fundamental objective for many wildlife population monitoring programs and ecological studies. Estimates of wildlife population abundance are imperative as they can be used to develop or apprise management practices and guidelines. However, mainly due to the fact that probability of detection of wildlife animals is typically less than one monitoring wildlife populations is challenging and costly. Survey sampling, taking repeated counts of individuals within a specific area, is often used to monitor populations as it is relatively lower in cost and effort compared to alternative sampling methods such as capture-recapture and removal sampling.

N-mixture models (Royle, 2004) are a class of hierarchical models that are very commonly used to estimate the absolute abundance of a species based on survey sampling. N-mixture models is popular as they provide a simple and cost effective way to estimate absolute abundance and account for imperfect detection by using data from unmarked individuals. Accounting for imperfect detection is vital for removing bias and producing reliable estimates of abundance and aiding inferences. In addition, N-mixture models also accommodates simultaneous inclusion of explanatory variables for both detection and abundance processes. Consequently, N-mixture models have been used for a range of purposes such as: “to evaluate the effectiveness of conservation actions (Romano et al., 2017), to better understand absolute abundance and population dynamics (Studds et al., 2017), to predict population responses to differing conservation scenarios (Ladin et al., 2016) and to forecast shifts in species distribution (Hunter et al., 2017)” Knappe et al. (2018).

This project is motivated by count data obtained by sampling yellow bellied toads (*Bombina variegata*) in the Italian Alps and the need to obtain an estimate of absolute abundance. This project aims to use new N-mixture models introduced by the researcher as well as existing N-mixture models to obtain an estimate of absolute abundance for the yellow bellied toads. The researcher introduces three new N-mixture models: Discrete Weibull - Binomial, Double Poisson - Binomial and Discrete Weibull Beta-Binomial.

However, a number of issues with N-mixture models have been found. Dennis et al. (2015) showed that when detection probability and the number of sampling occasions are small, infinite estimates of absolute abundance can arise. Barker et al. (2018) showed that by using a capture-recapture perspective, the loss of individual information resulting from count surveys compared to capture-recapture surveys is critical and causes parameter identifiability issues in the Binomial N-mixture model. Barker et al. (2018) also showed the inability of count data to discriminate among different hierarchical models which have vastly different implications on abundance. Through simulations, Knappe et al. (2018) showed that if the parametric assumptions of the N-mixture model are not exact, estimated abundance can be severely: underestimated if there is over-dispersion in the abundance process relative to the fitted model and overestimated if there is over-dispersion in the detection process. In a classical framework Kéry (2018) responded to these issues on parameter identifiability of Binomial N-mixture models and called for more research to be done on the parameter identifiability of Binomial N-mixture models.

Thus, this project aims to address some of the issues concerning parameter identifiability of N-mixture models using a Bayesian approach. In particular, this study aims to: investigate whether Wantanabe-Akaike information criterion (WAIC) can be used to discriminate among the different N-mixture models considered in this project, test whether detection probability in N-mixture models with Binomial observation process can be identified from count data, investigate whether one can convincingly discriminate between hierarchical representations of count data that have different implications on abundance and investigate whether one can unequivocally discriminate between N-mixture models.

The project is organised as follows: section 2 provides mathematical background on the different distributions used in this project, section 3 provides general background

information on N-mixture models, detailed description on the different N-mixture models considered in this project and description of software and methods used to fit the different N-mixture models. Section 4 describes the methods and techniques used to achieve the objectives of this research. Section 5 describes the analysis of the yellow bellied toads. Section 6 describes the simulation studies performed and the results obtained. Concluding remarking and future research is summarized in section 7.

2 Mathematical Background

2.1 Poisson distribution

The Poisson distribution is often the starting block used to model count data. Let y be a random variable that follows a Poisson distribution, then the probability mass function is defined as:

$$P(Y = y) = f(y; \lambda) = \frac{\lambda^y e^{-\lambda}}{y!}$$

for $y = 0, 1, \dots$ such that $\lambda > 0$ represents expected value(average). The Poisson distribution has the property

$$E(Y) = Var(Y) = \lambda.$$

2.2 Discrete Weibull (DW) Distribution

The Discrete Weibull (DW) distribution is the discrete form of the continuous Weibull distribution commonly used in survival analysis and failure time studies. There are three types of DW distribution. This project focuses on the most common type in the literature, the type 1 distribution of the DW distribution that has been developed by Nakagawa and Osaki (1975). Let y be a random variable that follows a (type 1) DW distribution, the cumulative distribution is defined as:

$$F(y; q, b) = \begin{cases} 1 - q^{(y+1)^b} & \text{for } y = 0, 1, 2, 3, \dots, \\ 0 & \text{otherwise} \end{cases}$$

and the probability mass function is defined as:

$$f(y; q, b) = \begin{cases} q^{y^b} - q^{(y+1)^b} & \text{for } y = 0, 1, 2, 3, \dots, \\ 0 & \text{otherwise} \end{cases}$$

where parameters q and b are defined such that $0 < q < 1$ and $b > 0$. Importantly, Kalktawi (2017) showed that the DW distribution can be used to model both over- and under-dispersed data. This feature enables the DW distribution to be a powerful distribution to model count data. With respect to the parameters (q, b) of the DW distribution, (Peluso et al., 2019) showed that when:

1. $0 < b \leq 1$ there is a case of over-dispersion, regardless of the value of q ,
2. $b \geq 3$ there is a case of under-dispersion, regardless of the value of q and
3. $1 < b < 3$ results in cases of both under-dispersion and over-dispersion depending on the value of q .

2.3 Negative Binomial (NB) distribution

The Negative Binomial (NB) distribution is often used to model over-dispersion in count data. Let y be a random variable that follows a NB distribution, then the probability mass function is defined as:

$$P(Y = y) = f(y; r, s) = \binom{y+r-1}{r-1} s^r (1-s)^y$$

for $y = 0, 1, \dots$ where $r > 0$ denotes the target for the number of successful trials and $0 < s \leq 1$ represents the probability of success in each trial.

2.4 Double Poisson (DP) distribution

The Double Poisson (DP) distribution, based on the double exponential family was first introduced by Efron (1986). Let y be a random variable that follows a DP distribution, the approximate probability mass function (pmf) is defined as:

$$P(Y = y) = f(y) = \theta^{1/2} e^{-\theta\lambda} \left(\frac{e^{-y} y^y}{y!} \right) \left(\frac{e\lambda}{y} \right)^{\theta y}$$

for $y = 0, 1, 2, \dots$, $\lambda > 0$ and $\theta > 0$. The exact DP density is defined as:

$$P(y = y) = \tilde{f}(y) = c(\lambda, \theta) f(y)$$

where $c(\lambda, \theta)$ represents the normalising constant that ensures the pmf sums to 1. This constant can be computed as

$$\frac{1}{c(\lambda, \theta)} = \sum_{y=0}^{\infty} f(y) \approx 1 + \frac{1-\theta}{12\lambda\theta} \left(1 + \frac{1}{\lambda\theta} \right).$$

The expected value and variance of $\tilde{f}(y)$ are estimated as:

$$E(Y) \approx \lambda \text{ and } Var(Y) \approx \frac{\lambda}{\theta}.$$

The DP distribution like the DW distribution provides the flexibility of modelling both under- and over-dispersed data. This flexibility is provided by the θ parameter of the DP such that $\theta > 1$ allows under-dispersion and $\theta < 1$ allows over-dispersion. In addition, when $\theta = 1$

$$\begin{aligned} f(y; \theta = 1) &= e^{-\lambda} \left(\frac{e^{-y} y^y}{y!} \right) \left(\frac{e\lambda}{y} \right)^y \\ &= e^{-\lambda} \frac{\lambda^y}{y!}. \end{aligned}$$

That is, when $\theta = 1$, the DP distribution reduces to the Poisson distribution. Hence, the DP N-mixture model is extremely flexible as it can be used to model over-, equi- and under-dispersion.

2.5 Zero inflated Poisson (ZIP) distribution

The zero inflated Poisson (ZIP) distribution is often used for count data that exhibit excess zeros. Using the ZIP distribution assumes that the count data consist of two types. The first type results in a Poisson distribution, which may be zero, whilst the second type always results in a zero count. Hence, the ZIP has two parameters, the mean of the Poisson distribution (λ) and the proportions of zero counts (ω). Let y be a random variable from the ZIP distribution, the probability mass function can then be written as:

$$P(Y = y) = \begin{cases} \omega + (1 - \omega)e^{-\lambda} & y = 0 \\ (1 - \omega)e^{-\lambda} \frac{\lambda^y}{y!} & y > 0. \end{cases}$$

such that $\lambda > 0$ and $0 < \omega \leq 1$.

3 Models

3.1 N-mixture Models

Using only replicated counts at multiple sites and assuming population closure, N-mixture models estimate absolute abundance and correct for imperfect detection. N-mixture models consist of two key processes: an abundance process describing the spatial variation in the number of individuals among sites and an detection process describing how many of the individuals present at each site are found at each visit. Count data are obtained during $j = 1, \dots, T$ sampling occasions at $i = 1, \dots, M$ sites. In N-mixture models, the counts (hereafter C_{ij}) are conditional dependent on the local abundance (hereafter N_i). For the abundance process, it is assumed that the local abundance at sites follow some discrete statistical distribution with probability function g . That is,

$$N_i \sim g(N, \lambda_i, \theta)$$

such that the draws from each sites are independent, λ_i represents the expected abundance at site i and θ represents an optional parameter for over-dispersion in the abundance process. To avoid over-parametrization, λ_i can be shared across all sites or can be expressed as a function of site specific covariates. Popular distributions for g are the Poisson, zero-inflated Poisson and Negative Binomial distributions. All of these distributions of g will be examined in this project as well as the lesser known distributions: Discrete Weibull and Double Poisson. Notably, this is the first study to the researcher's knowledge that the Discrete Weibull and Double Poisson distributions as abundance process of N-mixture models. These lesser known discrete distributions have the advantage of modelling both over and underdispersed data. In addition, in this project, λ_i was assumed to be constant across all sites.

In most N-mixture models, a Binomial distribution is used to describe the detection process. That is, it is assumed that:

$$C_{ij} \sim \text{Binomial}(N_i, p_{ij})$$

where N_i is the unknown local abundance and p_{ij} represents the probability of detecting an individual at site i and sampling occasion j . Similar to λ_i , p_{ij} can be assumed to be identical across sites and sampling occasions or can be expressed as a function of site specific covariates to avoid over parametrization. Thus, assuming N_i are independent random variables with discrete probability function $g(N, \lambda, \theta)$, the marginal likelihood for N-mixture models with Binomial detection processes can be written as:

$$L(p, \lambda, \theta; C_{ij}) = \prod_{i=1}^M \left\{ \sum_{N_i=\max_j C_{ij}}^{\infty} \left(\prod_{j=1}^T \text{Bin}(C_{ij}; N_i, p) \right) g(N_i, \lambda, \theta) \right\}. \quad (1)$$

This marginal likelihood considers all possible values for abundance at each site and in theory sums to infinity. However, in reality when N-mixture models are fitted using a classical approach, an upper bound needs to be chosen.

In addition, for N-mixture models with Binomial detection processes, the following key assumptions (Royle, 2015) are made:

1. *Closure assumption*: it is assumed that all the within-site variability of counts C is due to the detection probability p . That is, it is assumed that no deaths, births, emigration or immigration occurred in the study area during the sampling period.
2. *No false-positive errors*: it is assumed that only the target individuals are counted and that the same individual was not counted multiple times during a single occasion.
3. *Independence of detection*: given the Binomial distribution as the description of the observation process, it is assumed implicitly that individuals are detected independently of each other.
4. *Homogeneity of detection among N_i individuals*: as the N-mixture models uses count data, individual identification of individuals is not possible. Thus, it is implicitly assumed that all N_i individuals present at site i have an identical detection probability p_{ij} at sampling occasion j . In other words, it is assumed that there is no heterogeneity in detection probabilities among N_i .
5. *Parametric modelling assumptions*: the abundance process and the observation process of the Binomial N-mixture model make specific distributional assumptions. If these distributional assumptions are violated, biased estimates may result.

In order to obtain reliable estimates and inferences from an N-mixture model with Binomial detection process these assumptions have to hold. If one or more assumptions are violated biased estimates may result.

A Beta-Binomial distribution has also been used to model the detection process when detection probabilities are correlated (Martin et al., 2011). That is, it is assumed that:

$$C_{ij} \sim \text{Beta-Binomial}(N_i, p_{ij}, \rho)$$

where the detection probability at site i during survey j , p_{ij} , is defined such that:

$$p_{ij} \sim \text{Beta}(\alpha, \beta)$$

for $\alpha, \beta > 0$. To avoid over parametrisation, p_{ij} can be assumed to be constant over sites and sampling occasions which can then be approximated by the mean of the Beta distribution. In addition, ρ , which represents the correlation parameter is defined as

$$\rho = \frac{1}{\alpha + \beta + 1}.$$

Here, ρ , is viewed as a measure of correlation in individual behaviour or attributes of the site that could affect detection (Martin et al., 2011). It should be noted that correlation in individual behaviour or attributes of the site that could affect detection are not distinguishable when analysing data using a Beta-Binomial distribution as the detection process in N-mixture models. Martin et al. (2011) showed that the Beta-Binomial distribution as an detection process can be used to model correlated behaviour among individuals, thus, accounting for non-independent detection of individuals. Hence, by defining the detection process to be a Beta-Binomial distribution with detection probability that follows a beta distribution and a correlation parameter, correlated behaviour between individuals

and heterogeneity in detection probabilities can be accounted for in the detection process (Martin et al., 2011). For N-mixture models with Beta-Binomial detection processes the assumption of independence of detection and detection homogeneity are relaxed by considering a more flexible parametric distribution for the detection process.

N-mixture models with a Binomial or an Beta-Binomial detection process were also examined in this study. In both of these N-mixture models considered in this project p was assumed to be constant across sites and sampling occasions. For N-mixture models with Beta-Binomial detection processes p was approximated as the mean of the Beta distribution.

Table 1: N-mixture model models used in this project.

N-mixture model	Model for abundance process	Model for detection process
P-B	Poisson (λ)	Binomial(N, p)
DW-B	Discrete Weibull (q, b)	Binomial (N, p)
NB-B	Negative Binomial (r, s)	Binomial (N, p)
DP-B	Double Poisson (λ, θ)	Binomial (N, p)
ZIP-B	Zero inflated Poisson (λ, ω)	Binomial (N, p)
P-BB	Poisson (λ)	Beta - Binomial (N, p, ρ)
DW-B	Discrete Weibull (q, b)	Beta - Binomial (N, p, ρ)

Consequently, Table 1 displays the list of N-mixture models used in this project. The Poisson -Binomial (P-B) N-mixture model is the first N-mixture model developed by Royle (2004) and is one of the most commonly used N-mixture models. It accounts for imperfect detection by using a Binomial distribution as the detection process. As the DW distribution has the property of modelling both over- and under- dispersed data, the DW-B N-mixture model can be used to account for over- and under-dispersion present in the abundance process relative to the Poisson distribution. The NB-B N-mixture model can be used to account for over-dispersion in the abundance process relative to the Poisson distribution. The DP-B N-mixture model can be used to for over-, under- and equi-dispersion in the abundance process relative to the Poisson distribution. The P-BB N-mixture model can be used to account for heterogeneity in the detection process given there is no dispersion in the abundance process. The DW-BB N-mixture model can be used to account for over- and under-dispersion in the abundance process relative to the Poisson distribution and for correlated detection probabilities in the detection process.

3.2 Why use Bayesian inference on N-mixture Models?

In this study, N-mixture models were analysed in a Bayesian framework. The key reason is that in a Bayesian framework, probability is used directly to characterise uncertainty about model parameters and together with tools such as Markov chain Monte Carlo (MCMC) sampling methods, Bayesian analysis provides a flexible platform to model data and discover valuable information. MCMC is a class of sampling methods for drawing random samples from the target posterior distribution (Royle, 2015). By using MCMC sampling methods on N-mixture models, the latent variable (N_i) is preserved and the task of integrating the marginal likelihood is avoided by considering an infinite sum as shown in (1).

3.3 Software

The software used to code and perform Bayesian analysis on N-mixture models developed in this project is called NIMBLE. “NIMBLE is a system for building and sharing analy-

sis methods for statistical models, especially for hierarchical models and computationally-intensive methods” (NIMBLE Development Team, 2019). Many of the Bayesian software used for modelling data such as BUGS/JAGS are a black box that provides limited flexibility and are hard to expand. Thus, NIMBLE was developed to provide users with flexibility in model specification and algorithm programming in a Bayesian setting. “For model specification, NIMBLE retains BUGS compatibility and extends the BUGS programming language by creating model objects, which can manipulate variables, calculate log probability values, generate simulations, and query the relationships among variables” (de Valpine et al., 2017). NIMBLE also allows the user to create distributions that will be used and compiled in the BUGS code. For algorithm programming, NIMBLE provides a variety of standard algorithms such as MCMC, Monte Carlo expectation maximization (MCEM), etc. as well as allowing programmability of samplers of node(s) of a particular model (de Valpine et al., 2017). In addition, NIMBLE is a package in R, allowing users to operate in R. Hence, NIMBLE models and algorithms are programmed in R but are compiled in C++ for speed. This compilation in C++ makes NIMBLE significantly faster than programs such as BUGS/JAGS (NIMBLE Development Team, 2019).

3.3.1 Creating user-defined BUGS distribution in NIMBLE

NIMBLE enables users to create their own distributions and functions for usage in BUGS code. As a result, NIMBLE does not constrain users to the distributions or functions allowed in BUGS and JAGS. This ability enables users to create a wider range of models as compared to BUGS and JAGS. Functions and user-defined distributions are created as *nimbleFunctions* in NIMBLE. Writing *nimbleFunctions* entails declaring and correctly specifying the dimensionality of the arguments and the return object. Accordingly, to provide a user-defined BUGS distribution in NIMBLE, the probability density function and the simulation function must be programmed as *nimbleFunctions*. It is optional to provide percentile functions and quantile functions. Next, the user-defined distribution needs to be registered in NIMBLE.

3.3.2 Fitting N-mixture models

Using existing NIMBLE functions the P-B, NB-B and the P-BB N-mixture models were fitted. In fitting the DW-B N-mixture model, the DW distribution was programmed as a user defined distribution in NIMBLE. This was done by using the probability density function of the DW distribution to the programme the distribution function as a *nimbleFunction* and by using the method of inversion to programme the simulation function as a *nimbleFunction*. In constructing the DP-B N-mixture model, the DP distribution was programmed as a user defined distribution in NIMBLE. The R functions, *dDPO* and *rDPO* from the R package *gamlss.dist* were used to code the probability density function and simulation function of the Double Poisson distribution as *nimbleFunctions*. The ZIP-B N-mixture model was fitted by using existing code provided by (de Valpine, 2019) to program the ZIP distribution as a user defined distribution. Using the user defined distribution for the DW distribution together with existing NIMBLE functions, the DW-BB N-mixture model was fitted. Code for fitting each N-mixture model in NIMBLE is provided in Appendix A.

4 Model Checking

4.1 Posterior Predictive check

To assess the fit of N-mixture models used in this project, posterior predictive checks (Gelman et al., 2013) were used. In general, posterior predictive checks can be used to look at the difference between data simulated under a fitted model and observed data using graphical tools or by computing Bayesian p-values (Conn et al., 2018). If the observed data differs from the simulated data in a systematic way (e.g., increased variance), this indicates that the model assumptions are not met. The posterior predictive check procedure entails using an omnibus discrepancy function that compares the observed data to the simulated data. Omnibus discrepancy functions helps diagnose lack of fit. By using an omnibus discrepancy function, Bayesian p-values can be obtained by comparing the number of times the fit statistics for the simulated data are greater than the fit statistics for the observed data. A Bayesian p-value approximately equal to 0.5 indicates that the model fits the data well and that the model assumptions are valid. On the other hand, a Bayesian p-value close to 0 or 1 indicates that the model does not fit the data well and that one or more assumptions of the model are not satisfied. Therefore, in this study, Bayesian p-values were used to assess the fit and model assumptions of the N-mixture models considered in this project. The omnibus discrepancy function used to produce the Bayesian p-values in this project is the Chi-square (χ) statistic.

4.2 Predictive Accuracy

As complex models can be relatively easily built in a Bayesian setting, it is imperative to have measures that assess the fit of the models as well as measures that allow one to compare models. Popular tools such as posterior predictive checks and prior predictive checks can be used to evaluate the fit of a model. On the other hand, predictive accuracy measures can be used to compare models and to explore directions for possible improvements. Predictive accuracy measures simply compute the predictive accuracy for a model which can be used to evaluate the fit of a model as well as assess the forecast of a model. One key way of defining predictive accuracy or error of a model's prediction is by *probabilistic prediction* (Gelman et al., 2013). Consequently, to aid in the explanation of predictive measure in this section, suppose data $y_1, \dots, y_n$ are modelled as independent given parameters θ . Hence, let the likelihood be $p(y|\theta) = \prod_{i=1}^n p(y_i|\theta)$, $p(\theta)$ be the prior such that the posterior distribution is $p_{post}(\theta) = p(\theta|y)$ and the posterior predictive distribution is $p_{post}(\tilde{y}) = \int p(\tilde{y}_i|\theta)p_{post}(\theta)d\theta$ where $\tilde{y}_i$ denotes future data point i .

In probabilistic prediction, inferences about the predictive values are reported in such a way that the full uncertainty of the predictive values is taken into account. *Log-predictive density* or *log-likelihood* can be used as an measure of probabilistic prediction. Log-predictive density is simply $\log p(y|\theta)$ and is proportional to the mean square error if the model is normal with constant variance. It also has connections to the Kullback-Leibler information measure, such that, in the limit of large sample size, the model with the lowest Kullback-Leibler information measure will have the highest expected log predictive density resulting in the highest posterior probability (Gelman et al., 2013).

"The ideal measure of a model's fit would be its out-of-samples predictive performance for new data produced from the true data-generating process" Gelman et al. (2013). Hence, as defined by Gelman et al. (2013), let f be the true model and $\tilde{y}$ be the future data that could be seen. The out-of-sample predictive fit for a new data point $\tilde{y}_i$ using the logarithmic score is

$$\log p_{post}(\tilde{y}_i) = \log E_{post}(p(\tilde{y}_i|\theta)) = \log \int P(\tilde{y}_i)p_{post}(\theta) d\theta \quad (2)$$

In equation 2, $p_{post}(\tilde{y}_i)$ is the predictive density for $\tilde{y}_i$ induced by the posterior distribution $p_{post}(\theta)$. Consequently, as the future $\tilde{y}_i$ are not known, the expected log pointwise predicted density (elppd) can be used as a measure of predictive accuracy for n data points taken one at a time.

$$\text{Expected log pointwise predicted density (elppd)} = \sum_{i=1}^n E_{f_i}(\log p_{post}(\tilde{y}_i))$$

where $f_i(y)$ is the distribution representing the true data-generating process for y_i . However, as the parameter θ is not known, the log predictive density cannot be computed. Hence, the log pointwise predictive density (lppd) can be used to summarise the predictive accuracy of a fitted model.

$$\text{Log pointwise predictive density (lppd)} = \log \prod_{i=1}^n p_{post}(y_i) = \sum_{i=1}^n \log \int p(y_i|\theta) p_{post}(\theta) d\theta$$

Practically, lppd can be evaluated by using draws from the posterior distribution. Hence, let θ^s , for $s = 1, \dots, S$ be the draws from the posterior distribution, then the computed lppd ($\widehat{\text{lppd}}$) can be defined as:

$$\widehat{\text{lppd}} = \sum_{i=1}^n \log \left(\frac{1}{S} \sum_{s=1}^S p(y_i|\theta^s) \right)$$

4.2.1 Wantanable-Akaike information criterion (WAIC)

WAIC (introduced by Watanabe (2010)) also called “widely available information” criterion is a Bayesian approach for estimating the out-of-sample expectation by using the computed log pointwise posterior predictive density and by adding a bias adjustment, the effective number of parameters, to adjust for overfitting. Two estimates of the effective number of parameters have been proposed in the literature. The adjustments used in NIMBLE and thus in this project is as follows:

$$p_{\text{WAIC}} = \sum_{i=1}^n \text{var}_{post}(\log p(y_i|\theta)) \quad (3)$$

This adjustment can be computed by:

$$\text{computed } p_{\text{WAIC}} = \sum_{i=1}^n V_{s=1}^S(\log p(y_i|\theta_i))$$

where $V_{s=1}^S$ represents the sample variance. That is, equation (3) can be computed summing the posterior variance of the log predictive density over all data points y_i . Hence, the WAIC estimate of the out-of-sample expectation is:

$$\widehat{\text{elppd}}_{\text{WAIC}} = -2(\text{lppd} - p_{\text{WAIC}}) \quad (4)$$

This representation of WAIC allows it to be on the deviance scale. Hence, WAIC is comparable with Akaike information criterion (AIC), Deviance information criterion (DIC) and other measures of deviance. Therefore, a model with the lowest WAIC is comparatively the model with the best fit. Importantly, compared to AIC and DIC, WAIC has the advantage of averaging over the entire posterior distribution as opposed to conditioning on a single point. WAIC is particularly useful for hierarchical structures where the number of parameters increases with the sample size. However, WAIC relies on partitioning the data into n pieces, a task which might not be easy to do in structured data setting such as time series, spatial, and network data. Importantly, NIMBLE computes WAIC values for models from posterior samples produced by MCMC sampling methods. Thus, WAIC was used in this project to compare the N-mixture models considered.

4.3 Bias of Absolute Abundance Estimates

In this project, *bias* was used to compare absolute abundance posterior means to the true absolute abundance. Therefore, by following the computation of bias as outlined in Martin et al. (2011), the relative bias $\hat{B}_j$ of each estimator for each simulated data set j was computed as:

$$\hat{B}_j = \frac{N_{\text{total}_j} - N_j}{N_j}$$

where N_j represents the true absolute abundance and N_{total_j} represents the posterior mean absolute abundance for data set j . That is, $N_j = \sum_{i=1}^n N_{ij}$ where N_{ij} is the abundance for site i for simulated data set j . Consequently, if $\hat{B} > 0$, the estimator is positively biased (overestimates the abundance) whilst if $\hat{B} < 0$, the estimator is negatively biased (underestimates the abundance).

4.4 MCMC diagnostics

In order to assess mixing, convergence and autocorrelation of MCMC chains, both graphical and point estimates were used. Graphically, NIMBLE allows for the samples to be returned as an *coda* MCMC object. This allows for simultaneous use of R packages *coda* and *mcmcplots*. Hence, diagnostic plots such as trace, density and autocorrelations plots were used to investigate mixing, convergence and autocorrelation of MCMC chains. In addition, to assess convergence, Gelman and Rubin’s convergence diagnostic (Gelman et al., 1992), denoted here as $\hat{R}$ was used. $\hat{R}$ aims to assess convergence by comparing within-chain and between chain variance of multiple chains. $\hat{R}$ close to 1 signifies that the Markov chains have converged and sufficient posterior samples have been obtained. See Gelman et al. (2013) for detailed description of this method. Gelman et al. (1992) suggested the threshold of 1.2 for $\hat{R}$ to identify non-convergence of model parameters. Effective sample size (ESS) was also used to investigate autocorrelation. ESS is a statistic that adjusts the posterior sample size to account for autocorrelation. This adjusted sample size represents the independent samples from the posterior distribution. An ESS that is smaller relative to the number of samples of an Markov chain indicates high autocorrelation.

4.5 Model Identifiability

Non-identifiability is the scenario where models can be formulated from data without all the parameters of the model being estimable. “A model is said to be identifiable if no two values of the parameters give the same maximum likelihood for the data, while parameter redundant models can be re-expressed in terms of fewer than the original number of parameters (Catchpole and Morgan, 1997), resulting in likelihood surfaces with completely flat regions or surfaces.” Gimenez et al. (2009)

In a Bayesian setting, parameters that are non-identifiable are referred to as *weakly identifiable* (Garrett and Zeger, 2000). Weak identifiability occurs when the data provides little information about particular parameters resulting in the posterior and prior distributions being similar. Formally, let $f(\theta|Y)$ represent the marginal posterior distribution for data Y and parameter θ and let $f(\theta)$ represent the prior distribution for parameter θ . Then θ is said to be weakly identifiable when $f(\theta|Y) \approx f(\theta)$ (Garrett and Zeger, 2000). Consequently, with the increase in computational power, it is becoming easier to fit more complex Bayesian models using MCMC methods where inferences are made on posterior means/medians rather than modes. Thus, issues of model identifiability may not be apparent, making it is imperative to assess whether the parameters for a given model can be identified from a set of data. Importantly, Gimenez et al. (2009) highlighted some of the problems weak identifiability causes in a Bayesian setting. Two of the major problems attributed to weak identifiability of parameters are:

- By failing to recognise that a parameter is weakly identifiable, inaccurate conclusions may be formed on the non-identifiable parameter when there is no evidence beyond the prior distribution for that parameter (Garrett and Zeger, 2000);
- Strong correlations between parameters may be due to weak identifiability of parameters, resulting in poor mixing in the MCMC samples and slow convergence of MCMC chains (Rannala, 2002).

Equally important, it should be noted that Lindley (1971) highlighted that although a Bayesian model may be weakly identifiable, inferences made on the identifiable parameters of the model are valid. That is, valuable information is provided by the data on the identifiable parameters. Gimenez et al. (2009) proposed the following simple and effective model identifiability diagnostic tools:

- numerically evaluate and calibrate the overlap between each pair of marginal posterior and prior distributions;
- as proposed and used in Garrett and Zeger (2000), graphical plots can be generated to display the overlap between marginal posterior and prior distribution;
- investigate the correlation matrix between the parameters which can be easily obtained from MCMC samples of the parameters.

In order to compute the overlap between each univariate marginal posterior distribution and corresponding prior distribution, Garrett and Zeger (2000) proposed the quantity, τ_θ , which can be computed as

$$\tau_\theta = \int \min(f(\theta), f(\theta|Y)) d\theta. \quad (5)$$

Values of τ_θ lie within the interval $[0, 1]$, and when τ_θ is above the ad-hoc threshold 0.35 as proposed by Garrett and Zeger (2000), θ is weakly identifiable. It should be noted that Gimenez et al. (2009) found that this threshold is valid conditional on uniform prior distributions chosen for model parameters that are being tested for weak identifiability and if other priors distributions are chosen, it can be difficult to calibrate τ . Therefore, in this project the detection probabilities of N-mixtures models with Binomial observation processes were sensibly assumed to follow a $\text{Unif}(0,1)$ such that numerical computation as well as graphical representation of the overlap between marginal posterior and prior distributions can be computed to investigate whether the detection probabilities of N-mixtures models with Binomial observation processes can be identified from count data.

4.5.1 Practical Computation of τ

Following a similar route of computing equation 5 as proposed in Gimenez et al. (2009), the posterior distribution $f(\theta|Y)$ was estimated by means of a kernel density estimator $\hat{f}$ based on the MCMC generated values $x_i, i = 1, \dots, n$ where n was taken to be the last 1000 MCMC samples. Hence, $\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x-x_i}{h}\right)$ where K is a kernel function centered at the data points x_i and h is the bandwidth. The standard Gaussian kernel $K(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$ with its associated optimal bandwidth $h^* = 1.06\hat{\sigma}n^{-\frac{1}{5}}$ where $\hat{\sigma} = \min(\text{standard deviation}, \text{interquartile range}/1.34)$ (Silverman (2018), page 48) was used.

A sample from the distribution of the $\min$ function in the equation 5 can be obtained by calculating $y_i = \min(1, \hat{f}(x_i))$ for all i . Lastly, a Monte Carlo approximation to τ can be obtained by using

$$\hat{\tau} = \frac{\sum_i y_i \times (\max(x) - \min(x))}{n} \quad (6)$$

4.6 Savage Dickey density ratio (SDDR)

The Savage Dickey density ratio (SDDR) first published by Dickey and Lientz (1970) is a computational tool that can be used to produce the Bayes factor for hypothesis testing of nested models. Consider a hypothesis test where the null model, say H_0 , consists of the parameter vector $\theta = (\phi, \psi)$ such that $H_0 : \phi = \phi_0$ and $H_1 : \phi \neq \phi_0$ is the alternative hypothesis. Thus, H_0 is nested within H_1 where ϕ is the parameter of focus and ψ denotes the nuisance parameter vector. Nuisance parameters are parameters that both H_0 and H_1 have in common but are not of direct interest. Consequently, the SDDR computes the Bayes factor by: only considering H_1 , dividing the height of the posterior for ϕ by the height of the prior for ϕ , at the point of interest. Let D denote the observed data such that $p(D|H_0)$ and $p(D|H_1)$ represent the marginal likelihoods under H_0 and H_1 respectively, the SDDR is mathematically given as:

$$BF_{01} = \frac{p(D|H_0)}{p(D|H_1)} = \frac{p(\phi = \phi_0|D, H_1)}{p(\phi = \phi_0|H_1)}. \quad (7)$$

A mathematical proof of the SDDR can be found in appendix A of Wagenmakers et al. (2010). It should be noted that equation 7 holds when the prior for the nuisance parameter under H_1 equals the prior for the nuisance parameters under H_0 .

Key messages when using the SDDR are:

1. Relevance of the prior for the parameter of interest. The denominator of equation 7 represents the height of the prior for ϕ at $\phi = \phi_0$. Thus, the choice of prior can greatly impact the Bayes factor as a small prior height produces a large Bayes factor lending more support for H_0 and vice-versa. Therefore, the prior for the parameter of interest should be chosen carefully.
2. Irrelevance of the prior for nuisance parameters. For equation (7) to hold the prior of the nuisance parameter under H_1 must be equal the prior nuisance for the nuisance parameter under H_0 . Hence, nuisance parameter priors can be vague or improper as nuisance parameters are present in both models and cancels out in the computation of the Bayes factor.

4.6.1 Computation of the Savage Dickey density ratio

When the posterior distribution of ϕ has a known form, the height of the posterior can be computed and used directly in equation 7. On the other hand, when the posterior distribution of ϕ is not of known form, MCMC samples from the posterior distribution of ϕ can be used to approximate the the height of a uni-dimensional posterior distribution at the point of interest. Thus, following the density estimation in Wagenmakers et al. (2010), the logspline nonparametric density estimator proposed by Stone et al. (1997) was used in this project to approximate the posterior density of ϕ . A logspline density estimation uses a cubic spline to model a univariate log-density. The logspline density algorithm essentially adds and deletes knots to approximate the density where knot selection is based on AIC. Thus, logspline density estimation has the primary advantage of adding knots where they are most needed and deleting knots where they are not needed. Stone et al. (1997) provides more details on the logspline density algorithm. In addition, logspline density estimation performs well when the posterior distribution is confined and produces non-parametric estimates. That is, estimates that avoid assumptions about the form of the posterior distribution. Thus, by using the R package *polspline*, logspline density estimation was used to produce estimates of the heights of the posterior at the point of interest.

Importantly, the following limitations of the SDDR should be noted:

1. The SDDR can only be applied to nested models. That is, the SDDR cannot be used to compute the Bayes factor when the null hypothesis is not nested within the alternative hypothesis.
2. When the posterior is not in closed form, the SDDR depends on MCMC sampling methods to produce samples from the posterior.
3. As MCMC sampling methods are used to produce estimates from the posterior, convergence of the MCMC chains is important. If the chain has not converged, samples did not come from the posterior distribution. Thus, resulting in the SDDR producing misleading results.

5 Analysis of yellow bellied toads count data

During 2018, five populations of the yellow-bellied toad (*Bombina variegata*) were sampled in 27 sites represented by ponds or tanks located in different habitats, mainly vineyards and meadows, in the Italian Alps. Each site was sampled four times during the main part of the breeding season, from the end of May to the beginning of July. For this data set, the observed total population size was found to be 248 and the observed “relative abundance” was found to be 9.18. Observed total population size is the sum of the maximum counts at each site and Observed “relative abundance” is the observed total population size divided by the number of sampled sites.

Hence, count data on yellow-bellied toads were analysed using different N-mixture models considered in this study to obtain an estimate of the absolute abundance of yellow-bellied toads within the sampled region. Bayesian p-value (Bpv) was used to assess the fit of each N-mixture model and WAIC was used to compare the different N-mixture models based on fit. In addition, identifiability of detection probability in N-mixture models with Binomial detection processes were tested.

In the formulation of the different N-mixture models, the following priors were used: $p \sim \text{Unif}(0,1)$ was used for the detection probability in all N-mixture models with Binomial observation processes, $\lambda \sim \text{Gamma}(0.8, 0.08)$ was used for the parameter of the Poisson distribution of the P-B N-mixture model and for the parameter of the zero inflated Poisson distribution in the ZIP-B N-mixture model, $r \sim \text{Exp}(2)$ and $s \sim \text{Unif}(0,1)$ were used for the parameters of the NB in the NB-B N-mixture model, $q \sim \text{Unif}(0,1)$ and $b \sim \text{Exp}(1)$ were used for the parameters of the DW distribution in the DW-B N-mixture model, $\mu \sim \text{Exp}(0.5)$ and $\theta \sim \text{Exp}(0.5)$ was used for the parameters of the DP distribution in the DP-B N-mixture model, $\alpha \sim \text{Exp}(1)$, $\beta \sim \text{Exp}(1)$ were used for the parameters of the Beta distribution of p and $\lambda \sim \text{Gamma}(0.8,0.08)$ was used for the parameter of the Poisson distribution in the P-BB N-mixture model, and $\alpha \sim \text{Exp}(1)$, $\beta \sim \text{Exp}(1)$ were used for the parameters of the Beta distribution of p and $q \sim \text{Unif}(0,1)$ and $b \sim \text{Exp}(1)$ were used for the parameters of the DW distribution in the DW-BB N-mixture model.

Consequently, MCMC settings were used for each N-mixture model such that the absolute abundance estimate had an ESS around 1000 as well as model parameters having ESS greater than 1000. Table 2 displays the ESS obtained for the absolute abundance, model parameters and the number of thinning used for each model. During MCMC diagnostics, it was observed that both the DW-B and the NB-B N-mixture model displayed high levels of autocorrelation and poor mixing among the absolute abundance estimates. In order to account for the poor mixing and high levels on autocorrelation, thinning of 100 was tested. This did not improve the mixing and issue of autocorrelation. Hence, the following MCMC algorithms were tested: Automated Factor Slice Sampling (AFSS) (Tibbitts et al., 2014), which resulted in the following error: AFSS reached maximum number of iterations in atleast one dimension and Automated Parameter Blocking (Turek

and de Valpine et al., 2017), where it was found that the mixing and autocorrelation did not improve for either the DW-B and the NB-B N-mixtures. Hence, as can be seen from Table 2, high levels of thinning were used for the DW-B and NB-B N-mixture models, resulting in longer, computationally intensive MCMC chains. All other N-mixture models had moderate levels of autocorrelation and adequate mixing. In addition, it should be noted that all the N-mixture model parameters converged as they all produced $\hat{R} < 1.2$.

Table 2: ESS of absolute abundance (Ntotal) and model parameters for the different N-mixture models as well as the number of thinning used for each model.

Models	Parameters										thinning
	Ntotal	p	λ	q	b	θ	r	s	α	β	
P-B	1462	1306	3620	-	-	-	-	-	-	-	10
DW-B	961	1855	-	4130	9448	-	-	-	-	-	600
NB-B	967	1415	-	-	-	-	12030	1632	-	-	300
DP-B	1022	1039	3924	-	-	4947	-	-	-	-	30
ZIP-B	1218	1192	2256	-	-	-	-	-	-	-	10
P-BB	1138	1636	1152	-	-	-	-	-	5768	2230	40
DW-BB	1715	2225	-	4793	4913	-	-	-	1521	1068	40

Table 3: Results obtained for the yellow bellied count data using different N-mixture models. Ntotal represents the estimate of absolute abundance, Bpv represents the Bayesian p-value, WAIC denotes the Wantanabe-Akaike information criterion and τ_p represents the overlap between univariate marginal posterior distribution and prior distribution for p .

Models	Ntotal	p	Bpv	WAIC	τ_p
P-B	292	0.64	0	544.4	0.18
DW-B	1385	0.19	0.78	461.1	0.24
NB-B	1242	0.20	0.20	461.6	0.40
DP-B	457	0.41	0.61	463.5	0.32
ZIP-B	312	0.58	0	555.5	0.18
P-BB	1974	0.10	0.47	483.8	-
DW-BB	274	0.67	0.54	328.4	-

Table 3 displays the results obtained from fitting the different N-mixture models to the yellow bellied toad data. By looking at Table 3, it can be seen that the P-B and ZIP-B N-mixture have Bpv = 0. Hence, this provides evidence that the P-B and ZIP-B N-mixture models fit the data well. Furthermore, the dispersion parameter of the DW distribution, b was found to be 0.79 and the dispersion parameter of the DP distribution, θ was found to be 0.04. Hence, these values provide evidence that there is over-dispersion in the abundance process relative to the Poisson distribution as $0 < b \leq 1$ and $\theta < 1$ indicates over-dispersion. Importantly, by looking at Table 3, it can be seen that detection probability is identifiable in all N-mixture models with Binomial detection processes except the NB-B N-mixture model since $\tau > 35\%$. Hence, little information about detection probability is obtained from the NB-B N-mixture model.

Based on Bpv and WAIC, the DW-BB N-mixture model fitted the yellow bellied toad count data best. The Bpv of this model provides no evidence that the DW-BB N-mixture model does not fit the data well and the WAIC is significantly smaller than any other N-mixture model. Hence, it is reasonable to conclude that the absolute abundance of yellow bellied toads sampled in the Italian Alps is 274 with detection probability 0.67. Notably,

as can be seen from Table 3, the difference in absolute abundance estimates between the different N-mixture models is large. Hence, selecting the correct N-mixture model is imperative. As WAIC was used to select the DW-BB N-mixture model, a simulation study was carried out to investigate whether WAIC can be used to discriminate between the different N-mixture models considered in this project. This simulation study also investigates whether detection probability can be identified from N-mixture models with Binomial detection processes. Description and results are described in the next section.

6 Simulation

6.1 Identifiability of N-mixture models based on WAIC along with test for detection probability identifiability

To investigate whether WAIC can be used to discriminate between N-mixture models and whether detection probability can be identified from N-mixture models with Binomial detection processes, three simulation scenarios were preformed. Scenario 1: over-dispersion in the abundance process relative to the Poisson distribution with an Binomial detection process , scenario 2: heterogeneity in the the detection process and scenario 3: equi-dispersion in the abundance process relative to the Poisson distribution with an Binomial detection process.

In each scenario, 10 data sets were simulated with the number of sampling occasions $T = 5$, the number of sampled sites $M = 50$, and detection probability $p = 0.6$. Dennis et al. (2015) showed that when p is low, computational issues such as excessively high and even infinite estimates of absolute abundance occur regularly. Hence, p was chosen to be large to enable the direct comparison of the different types of N-mixture models without any computational issues. For scenario 1, data were simulated from the DW-B N-mixture model using the DW parameter values $q = 0.6$ and $b = 0.5$ to enable over-dispersion in the abundance process relative to the Poisson distribution. For scenario 2, data were simulated from the P-BB N-mixture model with the Poisson parameter $\lambda = 5$, Beta parameter $\alpha = 3$ and $\beta = 2$ to model heterogeneity in the detection process. For scenario 3, data were simulated from the P-B N-mixture model using the Poisson parameter $\lambda = 5$ to model equi-dispersion in the abundance process relative to the Poisson distribution.

Hence, data simulated under each scenario were fitted to the following N-mixture models: Discrete Weibull - Binomial (DW-B) , Double Poisson - Binomial (DP-B), Poisson - Binomial (P-B), Negative Binomial - Binomial (NB-B), zero inflated Poisson - Binomial (ZIP-B), Poisson - Beta Binomial (P-BB) and Discrete Weibull - Beta Binomial (DW-BB). N-mixture models were compared based on bias of absolute abundance estimate ($\text{Bias}_{\text{Ntotal}}$), Bayesian p-value (Bpv) and WAIC. In addition, identifiability of detection probability in N-mixture models with Binomial detection processes were examined by numerically evaluating the overlap (τ_p) between the respective marginal posterior and prior distributions.

For each simulated data, the seed of the random generator was set so that the same simulated under each scenario can be fitted to all of the N-mixture models considered in this study. In addition, it should be noted that the data simulated from these settings produced realistic observed "relative abundance". Observed "relative abundance" was computed as the observed total population size at the 50 sites divided by the number of sampled sites. In these simulations, the observed "relative abundance" was found to be around 2 to 6 which is similar to to the observed "relative abundance" in both the simulation study and the great tit example in chapter 6 of Royle (2015) respectively.

In each scenario, the following prior specifications were used: $p \sim \text{Unif}(0,1)$ was used as the prior for the detection probability of all N-mixture models with Binomial observation processes. For the DW-B N-mixture models, the following priors were used for

the parameters of the DW distribution: $q \sim \text{Unif}(0,1)$ and $b \sim \text{Exp}(1)$. For the DP-B N-mixture models, $\lambda \sim \text{Exp}(1)$ and $\theta \sim \text{Exp}(1)$ were used as the priors for the parameters of the DP distribution. For the P-B N-mixture models, $\lambda \sim \text{Gamma}(0.5,0.2)$ was used as the prior for the parameter of the Poisson distribution. For the NB-B N-mixture models, $r \sim \text{Exp}(0.5)$ and $s \sim \text{Unif}(0,1)$ priors were used for the parameters of the NB distribution. For the ZIP-B N-mixture models, $\lambda \sim \text{Unif}(0,5)$ was used as the prior of the parameter of the zero-inflated Poisson distribution. For the P-BB N-mixture models, $\alpha \sim \text{Exp}(1)$ and $\beta \sim \text{Exp}(1)$ priors were used for the parameters of the Beta distribution of p and $\lambda \sim \text{Unif}(0,5)$ prior was used for the parameter of the Poisson distribution. Lastly, for the DW-BB N-mixture models, $\alpha \sim \text{Exp}(1)$ and $\beta \sim \text{Exp}(1)$ priors were used for the parameters of the Beta distribution of p and $q \sim \text{Unif}(0,1)$, $b \sim \text{Exp}(1)$ priors were used for the parameter of the DW distribution.

Under each scenario, MCMC settings were used such that all nodes of each N-mixture models produced ESS of at least 1000. For scenario 1, during the MCMC diagnostic process of each N-mixture model, it was observed that the nodes of the DW-B, NB-B, P-BB and the DW-BB N-mixture models had higher levels of autocorrelation than the nodes of the P-B, ZIP-B and DP-B N-mixture models. Hence, for these models with higher levels of autocorrelation, longer MCMC chains with larger thinning were used. For scenario 2, the nodes of N-mixture models with Binomial detection processes produced higher levels of autocorrelation than N-mixture models with Beta-Binomial detection processes. Hence, longer MCMC chains with larger thinning were used for these models. For scenario 3, the nodes of all the N-mixture models except the ZIP-B were mixing well and displayed low levels of autocorrelation. Hence, longer chains were performed for the ZIP-B N-mixture models. Results obtained from carrying out this simulation study are displayed on Table 4. Importantly, it should be noted that for each simulation run under each scenario, all nodes of each N-mixture model converged according to $\hat{R}$.

Table 4: Simulation Results for all scenarios. $\text{Bias}_{\text{Ntotal}}$ represents the average bias of the absolute abundance estimate which was taken as the posterior mean and Bpv represents the average Bayesian p-values. These statistics represent the average values obtained by performing 10 simulations on the different N-mixture models.

Scenarios	N-mixture model	Bpv	$\text{Bias}_{\text{Ntotal}}$
1: Over-dispersion in abundance relative to the Poisson distribution.	P-B	0	-0.192
	DW-B	0.405	0.089
	NB-B	0.395	0.111
	DP-B	0.173	-0.024
	ZIP-B	0.013	-0.123
	P-BB	0.485	-0.215
	DW-BB	0.602	-0.142
2: Heterogeneity in detection process.	P-B	0.044	0.645
	DW-B	0.222	1.22
	NB-B	0.322	1.566
	DP-B	0.066	0.703
	ZIP-B	0.232	1.686
	P-BB	0.512	0.032
	DW-BB	0.514	0.033
3: Equi-dispersion in the abundance process relative to the Poisson distribution.	P-B	0.506	-0.0035
	DW-B	0.502	-0.0036
	NB-B	0.57	0.036
	DP-B	0.45	-0.015
	ZIP-B	0.98	0.546
	P-BB	0.724	-0.119
	DW-BB	0.596	-0.122

N-mixture models that best fit the data in each scenario were selected based on WAIC. When data was simulated under scenario 1, the P-BB N-mixture model was selected 10 times out of the 10 runs. In this scenario, the P-BB N-mixture WAIC values were smaller than WAIC value obtained from any other N-mixture models. When data was simulated under scenario 2, the DW-BB the P-BB N-mixture models were selected 8 and 2 times respectively of the 10 runs. In this scenario, the DW-BB and P-BB N-mixture WAIC values were within 2 units and were smaller than WAIC values for N-mixture models with Binomial detection processes in all runs. For scenario 3, the DW-BB and the P-BB N-mixture models were selected 6 and 4 times respectively of the 10 runs. In this scenario, the DW-BB and P-BB N-mixture WAIC values were also within 2 units of each other and smaller than WAIC values of N-mixture models with Binomial detection processes in all runs. Hence, when there was over-dispersion or equi-dispersion in the abundance process in relation to the Poisson distribution, WAIC failed to identify the true model in all runs. However, WAIC correctly identified the true model twice when there is heterogeneity in the detection process. It should also be noted that as can be see from Table 4, the N-mixture model selected under each scenario by WAIC as well as other N-mixture models have Bpv close to 0.5. This indicates that other N-mixture models as well as the N-mixture model selected by WAIC fit the data well. Thus, these result seems to suggest that WAIC is not a reliable tools for discriminating between N-mixture models as the results are mixed.

For scenarios 1 and 3, even tho the abundance process is generated under a model with over-dispersion or equi-dispersion relative to the Poisson distribution, N-mixture models with a Beta Binomial detection processes are favoured. However, as can be seen by looking at the $\text{Bias}_{\text{Ntotal}}$ column on Table 4, both the P-BB and the DW-BB N-mixture models

produced highly negative biased estimates of absolute abundance. Due to this results, it appears that using Beta Binomial N-mixture models when the detection process is in fact just a Binomial distribution can lead to negative bias in absolute abundance. Therefore, model selection of N-mixture models via WAIC should be treated with caution.

Furthermore, Knappe et al. (2018) recently showed in their simulations that not accounting for over-dispersion in the abundance process leads to strong underestimation of true absolute abundance and not accounting for heterogeneity in the detection process lead to strong over-estimation of the true absolute abundance. By looking at Table 4, it can be seen in scenario 1, N-mixture models that did not account for the over-dispersion in the abundance process strongly underestimated the true absolute abundance and in scenario 2, N-mixture models that did not account for heterogeneity in the detection process strongly over-estimated the true absolute abundance. Hence, these findings are consistent with the result obtained in Knappe et al. (2018). In addition, from Table 4, scenario 1, the average bias of absolute abundance estimates in the DW-BB N-mixture model is smaller than the P-BB N-mixture model, which suggests that accounting for over-dispersion in the abundance process helps a bit but the BB detection process pushes the estimates of absolute abundance to be underestimated.

For scenario 1, detection probability was identifiable in all runs for the P-B, ZIP-B and DP-B N-mixture models as it was found that $\tau_p < 35\%$. In this scenario, p was found to be weakly identifiable in the DW-B and the NB-B N-mixture models as 3 and 2 occasions respectively of the 10 runs $\tau_p > 35\%$. For scenario 2, detection probability was identifiable in all runs for the P-B, ZIP-B and DP-B N-mixture models. In this scenario, p was found to be weakly identifiable in the DW-B and the NB-B N-mixture models on 2 and 4 occasions respectively of the 10 runs. For scenario 3, p was found to be identifiable in all occasions for N-mixture models with Binomial detection processes. Hence, when data was simulated with high detection probability, over-dispersion present in the abundance process relative to the Poisson distribution or heterogeneity in the detection process, detection probability was found to be weakly identifiable in some occasions for both the DW-B and the NB-B N-mixture models. Hence, caution should be taken on the inference for detection probability in the DW-B and the NB-B N-mixtures when there is over-dispersion in the abundance process or heterogeneity in detection process. Regardless of over-dispersion present in the abundance process relative to the Poisson distribution or heterogeneity in the detection process, the P-B, ZIP-B and the DP-B N-mixture models provided evidence that count data provides valuable information on detection probability. Unfortunately, due to time constraints, identifiability of detection probability was not tested under the scenario where data was simulated with a small detection probability. This option should be considered to investigate whether identifiability of detection probability is dependent on the value of detection probability.

6.2 Can one discriminate among different hierarchical representations of count data?

In Barker et al. (2018), they pose an important question, can one discriminate among different hierarchical representations of count data ? They went on to answer this question by using simulation to compare two models under three scenarios.

Model 1: P-B N-mixture: $C_{ij}|N_i \stackrel{\text{iid}}{\sim} \text{Binomial}(N_i, p)$ where $N_i \stackrel{\text{iid}}{\sim} \text{Poisson}(\lambda)$

Model 2: Double Poisson Lognormal (DPLN): $C_{ij}|\lambda_i, \theta \stackrel{\text{iid}}{\sim} \text{Double Poisson}(\lambda_i, \theta)$ where $\lambda_i \stackrel{\text{iid}}{\sim} \text{Lognormal}(\mu, \sigma^2)$ with restriction $\theta = \mu/(\mu - \sigma^2)$.

These two models are quite different with local abundance(N_i) being featured in model 1

but not in model 2. Furthermore, in model 2, θ was constrained such that the first and second moments of the DPLN and P-B N-mixture model are identical. See Barker et al. (2018) supplementary materials for proof of this. Table 5 displays the three scenarios used in Barker et al. (2018) along with the restricted values of θ . Data was simulated across 50 sampling sites and 5 sampling occasion per site from both models under each scenario. The simulated data was fitted to both models where models were compared using AIC. The result produced is that one cannot clearly discriminate among competing model under this scenarios. It addition, it was also found that as the value of θ increased the ability to discriminate between these two models increased.

Table 5: Parameters used in simulations for the P-B N-mixture and the Double Poisson Lognormal models.

Scenario	N-mixture		Double poisson		Restricted
	λ	p	μ	σ^2	
A	30	0.1	3	0.3	1.1
B	30	0.3	9	2.7	1.4
C	30	0.5	15	7.5	2

Thus, in this project, the aim is to investigate whether one can discriminate among different hierarchical representations of count data, some of which do not provide inference on absolute abundance in a Bayesian setting. To investigate this, simulation was used to compare the P-B N-mixture model and the Double Poisson Lognormal model under the scenarios described in Table 5. The posterior predictive check, Bayesian p-value was used to assess the fit of the models whilst the predictive accuracy measure, WAIC, was used to compare the models.

Subsequently, due to time constraints, 5 data sets from 50 sites and 5 replicates per site were simulated under each scenarios in Table 5. Data simulated from the true model was fitted to both models in each scenario. This lead to six comparisons. Importantly, it should be noted that parameters μ and σ of the lognormal distribution are not the mean and standard deviation for a lognormal distributed variable, say X . They are the mean and standard deviation for the normally distributed natural logarithm, $\ln(X)$ (Wikipedia contributors, 2019). Thus, in order to simulated data from a lognormal distribution with mean μ and standard deviation σ , the R function *rlnorm* was used such that:

$$\text{meanlog} = \ln\left(\frac{\mu^2}{\sqrt{\sigma^2 + \mu^2}}\right)$$

$$\text{sdlog} = \sqrt{\ln\left(1 + \frac{\sigma^2}{\mu^2}\right)}$$

where meanlog and sdlog are the inputs for *rlnorm* and μ and σ^2 are the desired mean and variance for a lognormal data set.

The priors for λ and p of the N-mixture model were set as Gamma(30,1) and Unif(0,1) respectively all for scenarios. NIMBLE provides the lognormal distribution, *dlnorm*, which has parameters meanlog and tauolog where tauolog = $1/\text{sdlog}^2$. Let varlog = sdlog^2 , hence the following informative priors were used in the DPLN model: scenario A: meanlog $\sim$ Gamma(1,1), varlog $\sim$ Gamma(0.1,1); scenario B : meanlog $\sim$ Gamma(2,1), varlog $\sim$ Gamma(0.04,1) and scenario C: meanlog $\sim$ Gamma(2.7,1), varlog $\sim$ Gamma(0.04,1). In addition, in all scenarios for the both P-B N-mixture models and DPLN models, MCMC settings were used such that the ESS of all nodes of each models was atleast 1000. When

data were simulated from the P-B N-mixture model, more thinning and larger burning periods were used for the DPLN models and when data were simulated from the DPLN model, more thinning and larger burning periods were used for the P-B N-mixture models. Adopting such MCMC settings, it was found in all simulation runs in all scenarios, all nodes of the P-B N-mixture model and the DPLN model were mixing well, converged and had low levels of autocorrelation.

Carrying out such simulations, it was found that data simulated under the P-B N-mixture model, the DPLN model was either preferred or within 1 WAIC unit of the P-B N-mixture model in all 5 simulation runs for scenarios A and B and in 3 simulation runs for scenario C. When data was simulated from the DPLN model, the P-B N-mixture model was either preferred or within 1 WAIC unit of the DPLN model in all 5 simulation runs for scenario A, 4 simulation runs for scenario B and 2 simulation runs in scenario C. In addition, it was found for each simulation run under each scenario, the P-B N-mixture model and the DPLN model produced Bpv that indicated both models fitted the data well. Therefore, these close WAIC values and Bpv appears to represent an inability to convincingly discriminate between the P-B N-mixture model and the DPLN model under each scenario. It was also observed that as the value of θ increased, the ability to discriminate between these models increased. Thus, these results produced in this Bayesian setting suggest that under certain conditions one cannot unequivocally discriminate among different hierarchical representations of count data.

6.3 Identifiability of N-mixture models using Savage Dickey Density ratio

As it appears that the WAIC is not a reliable tool for discriminating between N-mixture models, another model selection approach, the Savage Dickey Density ratio (SDDR) was used to investigate whether one can unequivocally discriminate between N-mixture models. In particular, the SDDR was be used to test the hypothesis:

$$H_0 : \theta = 1 \quad \text{against} \quad H_1 : \theta \neq 1$$

where θ is the parameter of the Double Poisson distribution in the Double Poisson N-mixture model. When $\theta = 1$, the DP-B N-mixture model collapses into the P-B N-mixture model. Hence, by using the SDDR to produce the Bayes factor for the nested hypothesis test above, it can be investigated whether one can unequivocally discriminate between the P-B N-mixture model and the DP-B N-mixture model. Mathematically the Bayes factor (B_{01}) produced by the SDDR for this hypothesis test is:

$$BF_{01} = \frac{p(\theta = 1|D, H_1)}{p(\theta = 1|H_1)} = \frac{\text{height of the posterior for } \theta \text{ at } \theta = 1}{\text{height of the prior for } \theta \text{ at } \theta = 1}.$$

To answer this question, data were simulate from the DP distribution such that $\lambda = 5$, $\theta = (1.1, 4)$, $p = (0.2, 0.6)$, the number of sampled sites $M = 50$ and 5 sampling occasions at each site. This simulation setting results in four scenario: scenario I: $\theta = 1.1$, $p = 0.2$, scenario II: $\theta = 1.1$, $p = 0.6$, scenario III: $\theta = 4$, $p = 0.2$ and scenario IV: $\theta = 4$, $p = 0.6$. Notably, $\theta = 1.1$ was chosen as this value corresponds to the value of θ in scenario A on Table 5 in section 6.2 where it appears that according to WAIC, one cannot convincingly discriminate between different representations of count data and $\theta = 4$ was chosen to be quite large as there is some evidence that as θ increases, the ability to discriminate between model increases. Hence, these values of θ were chosen to investigate whether identifiability of the N-mixture models depends on the value of θ . On the other hand, these values of p were chosen to investigate whether detection probability affects the identifiability of N-mixture models. In addition, it should be noted that the number

of sampled sites and sampling occasions per site were chosen to match the Barker et al. (2018) settings used in section 6.2.

Consequently, under each scenarios, 5 data sets were simulated from the DP-B N-mixture model. Each data set was fitted to the DP-B N-mixture model to produce the respective Bayes factors in each scenario. Using the user defined distribution of the DP distribution, $p \sim \text{Unif}(0,1)$ and $\lambda \sim \text{Gamma}(5,1)$ priors were used for the detection probability and the λ parameter of the DP distribution respectively in all scenarios. In all scenarios, $\theta \sim \text{Gamma}(1,1)$ prior was used for the θ parameter of the DP distribution. In addition, for all scenarios, MCMC settings were chosen such that ESS of θ was around 1000. Table 7 displays the ESS obtained for θ in each scenario. Using such MCMC settings, it was found that all the nodes of the DP-B N-mixture models in each scenario mixed well and converged according to Gelman and Rubin’s convergence diagnostic. To apply the SDDR in each scenario, the last 1000 samples were drawn from the posterior distribution of θ to approximate the posterior density of θ . The posterior density of θ was approximated using logspline density estimation in each scenario. Using the respective logspline density estimates of θ , the heights of the posterior for θ at $\theta = 1$ were obtained and by using R’s built in function dgamma, the heights of the prior for θ at $\theta = 1$ were obtained to apply the SDDR in each scenario. For scenarios with $\theta = (1.1, 4)$, the data should have fair support of the true DP-B N-mixture model in order for one to unequivocally discriminate between N-mixture models. Jeffreys’s scale of evidence for Bayes factors, as shown in Table 6 was used to interpret the Bayes factor in each scenario. Hence, as shown on Table 6, if $BF_{01} < 1$, the data supports the true DP-B N-mixture model and if $BF_{01} > 1$, then the data supports the P-B N-mixture model. Table 7 displays the Bayes factor produced from applying the SDDR in all scenarios.

Table 6: Jeffreys’s scale of evidence for Bayes factors.

BF_{01}	Strength of evidence for supporting H_0
< 1	Negative (supports H_1)
1-3	Barely worth mentioning
3-10	Substantial
10-30	Strong
30-100	Very Strong
> 100	Decisive

Table 7: Results obtained from applying the SDDR in all scenarios. BF_{01} denotes the Bayes factor and ESS_{θ} represents the effective sample size obtained for θ .

	Scenarios	Simulation	BF_{01}	ESS_{θ}
I:	$\theta = 1.1, p = 0.2$	1	3.79	964
		2	2.09	906
		3	2.61	923
		4	3.05	984
		5	2.87	945
II:	$\theta = 1.1, p = 0.6$	1	1.74	1555
		2	4.9	1731
		3	0.58	1611
		4	3.55	1100
		5	5.21	954
III:	$\theta = 4, p = 0.2$	1	0.11	957
		2	0.8	940
		3	0.56	914
		4	0.08	970
		5	0.31	935
IV:	$\theta = 4, p = 0.6$	1	0	1629
		2	0.004	1121
		3	0	995
		4	0.002	1123
		5	0	973

As can be seen from Table 7, for scenario I, all $BF_{01} < 1$, for scenario II only once was $BF_{01} < 1$, for scenarios III and IV, all $BF_{01} < 1$. That is, in scenarios I and II, the data does not support the DP-B N-mixture model but in scenarios III and IV, the data supports the DP-B N-mixture model. Therefore, when the true value of θ is close 1 the SDDR does not support the true DP-B N-mixture model, agreeing with the findings by Barker et al. (2018). However, when the true value for θ is far from one, the SDDR can identify the true model, especially when p is large, as there is greater strength of the evidence for the true DP-B N-mixture model in scenario IV than in scenario III. Hence, these findings suggest that under certain circumstances, one can reliably discriminate between different N-mixture models.

7 Discussion

N-mixture models provide an attractive framework to gain inference on absolute abundance rather than relative abundance by using only replicated counts from unmarked individuals. Consequently, new N-mixture models introduced by the researcher as well as existing N-mixture models were fitted to count data on yellow belied toads to obtain an estimate of absolute abundance. It was found that the different N-mixture models produced hugely different estimates of absolute abundance. The WAIC was used to select the N-mixture model that fitted the data best. Subsequently, simulations were preformed to assess whether WAIC can be used to discriminate between the different N-mixture models considered in this project. Data were generated under three scenarios, scenario 1: over-dispersion in the abundance process, scenario 2: heterogeneity in the detection process and scenario 3: equi-dispersion in the abundance process. Only under scenario 2, in a small number of occasions did WAIC identify the true model. Hence, these result seems to suggest that WAIC is not a reliable tool for discriminating between N-mixture models

as the results were mixed. Thus, model selection of N-mixture models via WAIC should be treated with caution.

Importantly, it was found that N-mixture models with Beta-Binomial detection processes can be supported by WAIC even when the underlying truth is that there exists over-dispersion or equi-dispersion in abundance process. These highly supported Beta-Binomial N-mixture models also tend to underestimate absolute abundance in these case. Hence, these findings seem to suggest that the extra information provided by the heterogeneity in the detection process better accounts for the count data but fails to correctly estimate absolute abundance possibly due to the fact that count data does not provide any auxiliary data to correctly estimate detection probability when such a flexible model is considered for the detection process. This is plausible as Barker et al. (2018) advises if reliable estimates of absolute abundance are required, additional information such as recapture of marked individuals should be used for estimation of detection probability. Therefore, N-mixture models with Beta-Binomial detection processes should be used with caution and when the researcher has conclusive evidence of heterogeneity in detection process. Otherwise, misleading results may be produced. Consequently, more research should be carried out to understand why these results were obtained.

It was also found that N-mixture models that did not account for over-dispersion in the abundance process strongly underestimated absolute abundance and N-mixture models that did not account for heterogeneity in the detection process strongly overestimated the absolute abundance. These findings are consistent with Knappe et al. (2018) findings and are in agreement previous studies (Martin et al. (2011); Toribio et al. (2012)) which have showed that count data leads to N-mixture models being quite sensitive to small amounts of misspecification.

When count data was simulated with high detection probability, it was found that only in a small number of occasion was detection probability weakly identifiable. In particular, weak identifiability of p was found only a small number of times in the DW-B and the NB-B N-mixture models when there was over-dispersion in the abundance process or heterogeneity in the detection process. Detection probability was identifiable in all scenarios for all other N-mixture models considered. Hence, this provides evidence that N-mixture models with Binomial detection processes when fitted to count data with high detection probability can provide valuable information about detection probability. However, inference on detection probability in the DW-B and the NB-B N-mixture models should be made with caution. Unfortunately, due to time constraint, identifiability of detection probability in N-mixture models with Binomial observation processes could not be tested given data with low detection probability. Hence, future work should be carried out to tested whether detection probability in N-mixture models with Binomial detection processes are identifiable regardless of the value of detection probability of the data. In addition, future research should be preformed to develop better methods of assessing identifiability of N-mixture model parameters in a Bayesian setting.

Furthermore, following Barker et al. (2018) simulation settings and using Bayesian analysis, it was found that on the basic of model fit it seems that one cannot convincingly discriminate between different hierarchical representation of count data, some of which do not hold inference on abundance. This result is in agreement with the results obtained in Barker et al. (2018). However, by using SDDR to investigate whether one can unequivocally discriminate between N-mixture models, it appears that under certain scenarios, especially when detection probability is large, one can unequivocally discriminate between N-mixture models. Due to this result, future research should be carried out to potentially explore other methods to distinguish between N-mixture models and to better understand identifiability in N-mixture models.

However, in this project some limitations are worth noting. Due to time constraints,

the simulation results in section 6.1 were obtained using only 10 and in sections 6.2 and 6.3 only 5 simulated data sets were used. Although, this a reasonable number in a Bayesian setting, it is relatively low and results would be more reassuring if a longer simulation study that includes more data set is done. For simulations in section 6.1, if the researcher had more time, the researcher would preform simulations such that data would be simulated under each N-mixture models in this project and fitted to the different N-mixture models where WAIC would be recorded to assess whether WAIC can discriminate between the different N-mixture models. In addition, prior sensitivity analysis was not performed. Prior sensitivity analysis should be preformed on all prior settings in this study to determine how robust N-mixture models are to specification of priors chosen.

References

- Barker, R. J., Schofield, M. R., Link, W. A., and Sauer, J. R. 2018. On the reliability of n-mixture models for count data. *Biometrics*, 74(1):369–377.
- Catchpole, E. A. and Morgan, B. J. 1997. Detecting parameter redundancy. *Biometrika*, 84(1):187–196.
- Conn, P. B., Johnson, D. S., Williams, P. J., Melin, S. R., and Hooten, M. B. 2018. A guide to bayesian model checking for ecologists. *Ecological Monographs*, 88(4):526–542.
- de Valpine 2019. Adding a user-defined distribution to the bugs language. [Online; accessed 19-August-2019].
- de Valpine, P., Turek, D., Paciorek, C. J., Anderson-Bergman, C., Lang, D. T., and Bodik, R. 2017. Programming with models: writing statistical algorithms for general model structures with nimble. *Journal of Computational and Graphical Statistics*, 26(2):403–413.
- Dennis, E. B., Morgan, B. J. T., and Ridout, M. S. 2015. Computational aspects of n-mixture models. *Biometrics*, 71(1):237–246.
- Dickey, J. M. and Lientz, B. 1970. The weighted likelihood ratio, sharp hypotheses about chances, the order of a markov chain. *The Annals of Mathematical Statistics*, pages 214–226.
- Efron, B. 1986. Double exponential families and their use in generalized linear regression. *Journal of the American Statistical Association*, 81(395):709–721.
- Garrett, E. S. and Zeger, S. L. 2000. Latent class model diagnosis. *Biometrics*, 56(4):1055–1067.
- Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., and Rubin, D. B. 2013. *Bayesian data analysis*. Chapman and Hall/CRC.
- Gelman, A., Rubin, D. B., et al. 1992. Inference from iterative simulation using multiple sequences. *Statistical science*, 7(4):457–472.
- Gimenez, O., Morgan, B. J., and Brooks, S. P. 2009. Weak identifiability in models for mark-recapture-recovery data. In *Modeling demographic processes in marked populations*, pages 1055–1067. Springer.
- Hunter, E., Nibbelink, N., and Cooper, R. 2017. Divergent forecasts for two salt marsh specialists in response to sea level rise. *Animal Conservation*, 20(1):20–28.

- Kalktawi, H. S. 2017. *Discrete Weibull regression model for count data*. PhD thesis, Brunel University London.
- Kéry, M. 2018. Identifiability in n-mixture models: A large-scale screening test with bird data. *Ecology*, 99(2):281–288.
- Knape, J., Arlt, D., Barraquand, F., Berg, Å., Chevalier, M., Pärt, T., Ruete, A., and Żmihorski, M. 2018. Sensitivity of binomial n-mixture models to overdispersion: The importance of assessing model fit. *Methods in Ecology and Evolution*, 9(10):2102–2114.
- Ladin, Z. S., D’Amico, V., Baetens, J. M., Roth, R. R., and Shriver, W. G. 2016. Predicting metapopulation responses to conservation in human-dominated landscapes. *Frontiers in Ecology and Evolution*, 4:122.
- Lindley, D. 1971. Bayesian statistics: A review. philadelphia. *Society for Industrial and Applied Mathematics*.
- Martin, J., Royle, J. A., Mackenzie, D. I., Edwards, H. H., Kery, M., and Gardner, B. 2011. Accounting for non-independent detection when estimating abundance of organisms with a bayesian approach. *Methods in Ecology and Evolution*, 2(6):595–601.
- Nakagawa, T. and Osaki, S. 1975. The discrete weibull distribution. *IEEE Transactions on Reliability*, 24(5):300–301.
- NIMBLE Development Team 2019. Nimble user manual. R package manual version 0.8.0.
- Peluso, A., Vinciotti, V., and Yu, K. 2019. Discrete weibull generalized additive model: an application to count fertility data. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 68(3):565–583.
- Rannala, B. 2002. Identifiability of parameters in mcmc bayesian inference of phylogeny. *Systematic Biology*, 51(5):754–760.
- Romano, A., Costa, A., Basile, M., Raimondi, R., Posillico, M., Roger, D. S., Crisci, A., Piraccini, R., Raia, P., Matteucci, G., et al. 2017. Conservation of salamanders in managed forests: Methods and costs of monitoring abundance and habitat selection. *Forest ecology and management*, 400:12–18.
- Royle, J. A. 2004. N-mixture models for estimating population size from spatially replicated counts. *Biometrics*, 60(1):108–115.
- Royle, M. K. J. 2015. *Applied hierarchical modeling in ecology : analysis of distribution, abundance and species richness in R and BUGS. Volume 1, Prelude and static models*. Academic Press.
- Silverman, B. W. 2018. *Density estimation for statistics and data analysis*. Routledge.
- Stone, C. J., Hansen, M. H., Kooperberg, C., Truong, Y. K., et al. 1997. Polynomial splines and their tensor products in extended linear modeling: 1994 wald memorial lecture. *The Annals of Statistics*, 25(4):1371–1470.
- Studds, C. E., Kendall, B. E., Murray, N. J., Wilson, H. B., Rogers, D. I., Clemens, R. S., Gosbell, K., Hassell, C. J., Jessop, R., Melville, D. S., et al. 2017. Rapid population decline in migratory shorebirds relying on yellow sea tidal mudflats as stopover sites. *Nature communications*, 8:14895.
- Tibbits, M. M., Groendyke, C., Haran, M., and Liechty, J. C. 2014. Automated factor slice sampling. *Journal of Computational and Graphical Statistics*, 23(2):543–563.

- Toribio, S., Gray, B., and Liang, S. 2012. An evaluation of the bayesian approach to fitting the n-mixture model for use with pseudo-replicated count data. *Journal of Statistical Computation and Simulation*, 82(8):1135–1143.
- Turek, D. and de Valpine et al. 2017. Automated parameter blocking for efficient markov chain monte carlo sampling. *Bayesian Analysis*, 12(2):465–490.
- Wagenmakers, E.-J., Lodewyckx, T., Kuriyal, H., and Grasman, R. 2010. Bayesian hypothesis testing for psychologists: A tutorial on the savage–dickey method. *Cognitive psychology*, 60(3):158–189.
- Watanabe, S. 2010. Asymptotic equivalence of bayes cross validation and widely applicable information criterion in singular learning theory. *Journal of Machine Learning Research*, 11(Dec):3571–3594.
- Wikipedia contributors 2019. Log-normal distribution — Wikipedia, the free encyclopedia. [Online; accessed 19-August-2019].

Appendices

A Fitting N-mixture models in NIMBLE

P-B N-mixture Model

```
library(nimble)

win.data = list(DWC = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)
str(win.data)

# Specify model in BUGS language:
N_mixcode <- nimbleCode( {
  # Priors
  lambda ~ dgamma(0.5, 0.2)
  p ~ dunif(0, 1)
  #p ~ dbeta(4,2)

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ dpois(lambda)      # State model

    # Observation model for replicated counts
    for (j in 1:J) {
      DWC[i,j] ~ dbin(p, N[i]) # Observation model
    }
  }
}

# Posterior predictive distributions of chi2 discrepancy
for (i in 1:M) {
  for (j in 1:J) {
```

```

    C.sim[i,j] ~ dbin(p, N[i]) # Create new data set under model
    e.count[i,j] <- N[i] * p # Expected datum
    # Chi-square discrepancy for the actual data
    chi2.actual[i,j] <- pow((DWC[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
    # Chi-square discrepancy for the simulated ('perfect') data
    chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
    # Add small value e to denominator to avoid division by zero
  }
}
# Add up individual chi2 values for overall fit statistic
fit.actual <- sum(chi2.actual[1:M, 1:J]) # Fit statistic for actual data set
fit.sim <- sum(chi2.sim[1:M, 1:J]) # Fit statistic for a fitting model
bpv <- step(fit.sim-fit.actual) # Bayesian p-value
c.hat <- fit.actual/fit.sim # c-hat estimate
Ntotal <- sum(N[1:M]) # Total population size at M sites combined

})

Nst <- apply(DWC, 1, max)+1
Bmodel = nimbleModel(N_mixcode, constants = win.data, inits =
  list(N = Nst, lambda = rgamma(1,0.5,0.2) , p =runif(1,0,1)))
cBmodel <- compileNimble(Bmodel)

Bconf = configureMCMC(Bmodel)
Bconf$monitors
Bconf$addMonitors(c("Ntotal", "fit.actual", "fit.sim", "c.hat", "bpv",
  "lambda", "p","N"))
Bconf$getSamplers()

Bmcmc = buildMCMC(Bconf, enableWAIC = TRUE)
Bmod = compileNimble(Bmcmc, project = Bmodel, resetFunctions = TRUE)
Bmcmc.out = runMCMC(Bmod, niter = 24000, nburnin = 4000, thin = 10,
  samplesAsCodaMCMC = TRUE, nchains = 2, WAIC = TRUE, summary = TRUE)

```

DW-B N-mixture model

```

library(nimble)
# creating user defined DW distribution
dimydwibull <- nimbleFunction(
  run = function(x = double(0), q = double(0), b = double(0),
    log = integer(0, default= 0)){
    returnType(double(0))
    logProb = log(q^(x^b) - q^((x+1)^b))
    if(log){
      return(logProb)
    }
    else{
      return(exp(logProb))
    }
  }
}

```

```

)

r1mydweibull <- nimbleFunction(
  run = function(n = double(0) ,q = double(0), b = double(0)){
    returnType(double(0))
    if(n != 1) print("r1mydweibull only allows n = 1; using n = 1.")
    u <- runif(1, min =0, max =1)
    s <- ceiling((log(1 - u)/log(q))^(1/b)) - 1
    return(s)
  }
)

registerDistributions(list(
  d1mydweibull = list(
    BUGSdist = "d1mydweibull(q,b)",
    discrete = TRUE,
    range = c(0, Inf),
    types = c('value = integer()', 'q = double()', 'b = double()')
  )))

win.data = list(DWC = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)
str(win.data)

N_mixdwb <- nimbleCode( {

  # Priors
  p ~ dunif(0,1)
  q ~ dunif(0,1) # 0<q<1
  b ~ dexp(1)

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ d1mydweibull(q,b) # State model

    # Observation model for replicated counts
    for (j in 1:J) {
      DWC[i,j] ~ dbin(p, N[i]) # Observation model
    }
  }

  for (i in 1:M) {
    for (j in 1:J) {
      C.sim[i,j] ~ dbin(p, N[i]) # Create new data set under model
      e.count[i,j] <- N[i] * p # Expected datum
      # Chi-square discrepancy for the actual data
      chi2.actual[i,j] <- pow((DWC[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Chi-square discrepancy for the simulated ('perfect') data
      chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Add small value e to denominator to avoid division by zero
    }
  }
}

```



```

}
# Add up individual chi2 values for overall fit statistic
fit.actual <- sum(chi2.actual[1:M, 1:J]) # Fit statistic for actual data set
fit.sim <- sum(chi2.sim[1:M, 1:J])      # Fit statistic for a fitting model
bpv <- step(fit.sim-fit.actual)        # Bayesian p-value
c.hat <- fit.actual/fit.sim             # c-hat estimate
Ntotal <- sum(N[1:M])                  # Total population size at M sites combined

})

Nst <- apply(DWC, 1, max) +1           # Avoid data/model/inputs conflict

DWBmodel = nimbleModel(N_mixdwb, constants = win.data,
  inits = list(N = Nst, q = runif(1,0,1) , b=rgamma(1,1), p = runif(1,0,1)))
cDWBmodel <- compileNimble(DWBmodel)

DWBconf = configureMCMC(DWBmodel)
DWBconf$monitors
DWBconf$addMonitors(c("Ntotal", "fit.actual", "fit.sim", "c.hat",
  "bpv", "q", "b" , "p", "N"))

DWBmcmc = buildMCMC(DWBconf, enableWAIC= TRUE)
DWB = compileNimble(DWBmcmc, project = DWBmodel, resetFunctions = TRUE)
DWBmcmc.out = runMCMC(DWB, niter = 120000, nburnin = 4000, thin = 40,
  samplesAsCodaMCMC = TRUE, nchains = 2, WAIC = TRUE, summary = TRUE)
DWBmcmc.out$WAIC
summary = DWBmcmc.out$summary$all.chains[c('Ntotal', 'fit.actual',
  'fit.sim', 'c.hat', 'bpv', 'q', 'b' , 'p'),]
summary

```

NB-B N-mixture model

```

library(nimble)
win.data = list(DWC = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)
str(win.data)

N_mixcode_NB <- nimbleCode( {
  # Priors

  r ~ dexp(0.25)
  p ~ dunif(0,1)
  p1 ~ dunif(0,1)

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ dnegbin(p1,r)      # State model
    # Some intermediate derived quantities

    # Observation model for replicated counts

```

```

    for (j in 1:J) {
      DWC[i,j] ~ dbin(p, N[i]) # Observation model
    }
  }

# Posterior predictive distributions of chi2 discrepancy
for (i in 1:M) {
  for (j in 1:J) {
    C.sim[i,j] ~ dbin(p, N[i]) # Create new data set under model
    e.count[i,j] <- N[i] * p # Expected datum
    # Chi-square discrepancy for the actual data
    chi2.actual[i,j] <- pow((DWC[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
    # Chi-square discrepancy for the simulated ('perfect') data
    chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
    # Add small value e to denominator to avoid division by zero
  }
}

# Add up individual chi2 values for overall fit statistic
fit.actual <- sum(chi2.actual[1:M, 1:J]) # Fit statistic for actual data set
fit.sim <- sum(chi2.sim[1:M, 1:J]) # Fit statistic for a fitting model
bpv <- step(fit.sim-fit.actual) # Bayesian p-value
c.hat <- fit.actual/fit.sim # c-hat estimate
Ntotal <- sum(N[1:M]) # Total population size at M sites combined

})

Nst <- apply(DWC, 1, max)+1 # Avoid data/model/inits conflict
Nst
NBmodel = nimbleModel(N_mixcode_NB, constants = win.data,
inits = list(N = Nst, p =runif(1,0,1), r=rexp(1,10) ))
cNBmodel <- compileNimble(NBmodel)
NBconf = configureMCMC(NBmodel)
NBconf$monitors
NBconf$addMonitors(c("r","Ntotal", "fit.actual", "fit.sim", "c.hat",
"bpv", "p","N"))

NBmcmc = buildMCMC(NBconf, enableWAIC = TRUE)
NBmod = compileNimble(NBmcmc, project = NBmodel, resetFunctions = TRUE)
NBmcmc.out = runMCMC(NBmod, niter = 200000, nburnin = 4000, thin = 80,
samplesAsCodaMCMC = TRUE, nchains = 2, WAIC = TRUE, summary = TRUE)
NBmcmc.out$WAIC
nbsummary = NBmcmc.out$summary$all.chains[c('Ntotal', 'fit.actual',
'fit.sim', 'c.hat', 'bpv', 'p', 'r','p1'),]
nbsummary

```

DP-B N-mixture model

```

library(nimble)
library(gamlss.dist)

```

```

myget_C= nimbleRcall(function(x=double(),mu = double(),sigma =
  double()){}, Rfun = 'get_C', returnType = double(), envir = .GlobalEnv )

dmyDPO <- nimbleFunction(
  run = function(x= double(), mu = double(), sigma= double(),
  log = integer(0, default= 0)){
    returnType(double(0))
    if(x==0){
      logofx <- 1
      lh <- -0.5 * log(sigma) - (mu/sigma) - lgamma(x + 1) + x *
        logofx - x + (x * log(mu))/sigma + x/sigma - (x * logofx)/sigma +
        myget_C(x, mu, sigma)
    } else {
      logofx <- log(x)
      lh <- -0.5 * log(sigma) - (mu/sigma) - lgamma(x + 1) + x *
        logofx - x + (x * log(mu))/sigma + x/sigma - (x * logofx)/sigma +
        myget_C(x, mu, sigma)
    }
    if (log){
      return(lh)
    }
    else return(exp(lh))
  }
)

rdpo = function(n, mu, sigma,max.value = 10000){
  if(n != 1) print("rmyDPO only allows n = 1; using n = 1.")
  else{
    p <- runif(1)
    r <- qDPO(p, mu = mu, sigma = sigma, max.value = max.value)
    r
  }
}

rmyDPO = nimbleRcall(function(n=double(),mu = double(),
sigma = double()){}, Rfun = 'rdpo', returnType = double(), envir = .GlobalEnv )

registerDistributions(list(
  dmyDPO = list(
    BUGSdist = "dmyDPO(mu,sigma)",
    discrete = TRUE,
    range = c(0, Inf),
    types = c('value = integer()', 'mu = double()', 'sigma = double()')
  )))

win.data = list(C = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)

```

```

str(win.data)

# Specify model in BUGS language:
N_mixcode <- nimbleCode( {
  # Priors
  mu ~ dexp(1)
  sigma ~ dexp(1)
  p ~ dunif(0, 1)

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ dmyDPO(mu,sigma)    # State model

    # Observation model for replicated counts
    for (j in 1:J) {
      C[i,j] ~ dbin(p, N[i]) # Observation model
    }
  }

  # Posterior predictive distributions of chi2 discrepancy
  for (i in 1:M) {
    for (j in 1:J) {
      C.sim[i,j] ~ dbin(p, N[i]) # Create new data set under model
      e.count[i,j] <- N[i] * p # Expected datum
      # Chi-square discrepancy for the actual data
      chi2.actual[i,j] <- pow((C[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Chi-square discrepancy for the simulated ('perfect') data
      chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Add small value e to denominator to avoid division by zero
    }
  }

  # Add up individual chi2 values for overall fit statistic
  fit.actual <- sum(chi2.actual[1:M, 1:J]) # Fit statistic for actual data set
  fit.sim <- sum(chi2.sim[1:M, 1:J]) # Fit statistic for a fitting model
  bpv <- step(fit.sim-fit.actual) # Bayesian p-value
  c.hat <- fit.actual/fit.sim # c-hat estimate

  Ntotal <- sum(N[1:M]) # Total population size at M sites combined
})

Nst <- apply(DWC, 1, max)+1 # Avoid data/model/inputs conflict
Nst

DPmodel = nimbleModel(N_mixcode, constants = win.data,
  inits = list(N = Nst, mu =runif(1,0.5,1.5), sigma =runif(1,0.5,1.5)))
cDPmodel <- compileNimble(DPmodel)

DPconf = configureMCMC(DPmodel)
DPconf$monitors
DPconf$addMonitors(c("Ntotal", "fit.actual", "fit.sim", "c.hat", "bpv",

```

```

"mu","sigma", "p","N"))
DPmcmc = buildMCMC(DPconf,enableWAIC= TRUE)

DPmod = compileNimble(DPmcmc, project = DPmodel, resetFunctions = TRUE)
DPmcmc.out = runMCMC(DPmod, niter = 44000, nburnin = 4000,thin = 10,
  samplesAsCodaMCMC = TRUE, nchains = 1, WAIC = TRUE, summary = TRUE)
DPmcmc.out$WAIC
DPsummary = DPmcmc.out$summary$all.chains[c('Ntotal', 'fit.actual',
  'fit.sim', 'c.hat', 'bpv', 'mu','sigma' , 'p'),]
DPsummary
DPmcmc.out$summary[c('Ntotal', 'fit.actual', 'fit.sim', 'c.hat', 'bpv',
  'mu','sigma' , 'p'),]

```

ZIP-B N-mixture model

```

library(nimble)
dZIP <- nimbleFunction(
  run = function(x = integer(), lambda = double(),
    zeroProb = double(), log = logical(0, default = 0)) {
    returnType(double())
    ## First handle non-zero data
    if(x != 0) {
      ## return the log probability if log = TRUE
      if(log) return(dpois(x, lambda, log = TRUE) + log(1-zeroProb))
      ## or the probability if log = FALSE
      else return((1-zeroProb) * dpois(x, lambda, log = FALSE))
    }
    ## From here down we know x is 0
    totalProbZero <- zeroProb + (1-zeroProb) * dpois(0, lambda, log = FALSE)
    if(log) return(log(totalProbZero))
    return(totalProbZero)
  })

rZIP <- nimbleFunction(
  run = function(n = integer(), lambda = double(), zeroProb = double()) {
    returnType(integer())
    isStructuralZero <- rbinom(1, prob = zeroProb, size = 1)
    if(isStructuralZero) return(0)
    return(rpois(1, lambda))
  })

registerDistributions(list(
  dZIP = list(
    BUGSdist = "dZIP(lambda, zeroProb)",
    discrete = TRUE,
    range = c(0, Inf),
    types = c('value = integer()', 'lambda = double()', 'zeroProb = double()')
  )))

win.data = list(DWC = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)
str(win.data)

```

```

# Specify model in BUGS language:
ZIP_mixcode <- nimbleCode( {
  # Priors
  lambda ~ dgamma(0.5,0.2)
  p ~ dunif(0, 1)

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ dZIP(lambda, zeroProb = p)      # State model

    # Observation model for replicated counts
    for (j in 1:J) {
      DWC[i,j] ~ dbin(p, N[i]) # Observation model
    }
  }

  # Posterior predictive distributions of chi2 discrepancy
  for (i in 1:M) {
    for (j in 1:J) {
      C.sim[i,j] ~ dbin(p, N[i]) # Create new data set under model
      e.count[i,j] <- N[i] * p # Expected datum
      # Chi-square discrepancy for the actual data
      chi2.actual[i,j] <- pow((DWC[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Chi-square discrepancy for the simulated ('perfect') data
      chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Add small value e to denominator to avoid division by zero
    }
  }

  # Add up individual chi2 values for overall fit statistic
  fit.actual <- sum(chi2.actual[1:M, 1:J]) # Fit statistic for actual data set
  fit.sim <- sum(chi2.sim[1:M, 1:J]) # Fit statistic for a fitting model
  bpv <- step(fit.sim-fit.actual) # Bayesian p-value
  c.hat <- fit.actual/fit.sim # c-hat estimate
  Ntotal <- sum(N[1:M]) # Total population size at M sites combined

})

Nst <- apply(DWC, 1, max)+1
ZIPmodel = nimbleModel(ZIP_mixcode, constants = win.data,
  inits = list(N = Nst, lambda =runif(1,0.5,1.5), p =runif(1,0,1)))
cZIPmodel <- compileNimble(ZIPmodel)

ZIPconf = configureMCMC(ZIPmodel)
ZIPconf$monitors
ZIPconf$addMonitors(c("Ntotal", "fit.actual", "fit.sim", "c.hat",
  "bpv", "lambda", "p","N"))

```

```

ZIPmcmc = buildMCMC(ZIPconf, enableWAIC = TRUE)
ZIPmod = compileNimble(ZIPmcmc, project = ZIPmodel, resetFunctions = TRUE)
ZIPmcmc.out = runMCMC(ZIPmod, niter = 35000, nburnin = 4000,
thin = 10, samplesAsCodaMCMC = TRUE, nchains = 2, WAIC = TRUE, summary = TRUE)
ZIPmcmc.out$WAIC
ZIPsummary = ZIPmcmc.out$summary$all.chains[c('Ntotal', 'fit.actual',
'fit.sim', 'c.hat', 'bpv', 'p', 'lambda'),]
ZIPsummary

```

P-BB N-mixture model

```

library(nimble)
win.data = list(DWC = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)
str(win.data)

PBB_mixcode <- nimbleCode( {
  # Priors
  alpha ~dexp(1)
  beta ~dexp(1)
  lambda ~ dgamma(0.5,0.2)

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ dpois(lambda)      # State model

    # Observation model for replicated counts
    for (j in 1:J) {
      DWC[i,j] ~ dbin(p[i,j], N[i]) # Observation model
      p[i,j] ~ dbeta(alpha, beta)
      #detection probability p follows a beta distribution
    }
  }
  # Posterior predictive distributions of chi2 discrepancy
  for (i in 1:M) {
    for (j in 1:J) {
      C.sim[i,j] ~ dbin(p[i,j], N[i]) # Create new data set under model
      e.count[i,j] <- N[i] * p[i,j]   # Expected datum
      # Chi-square discrepancy for the actual data
      chi2.actual[i,j] <- pow((DWC[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Chi-square discrepancy for the simulated ('perfect') data
      chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Add small value e to denominator to avoid division by zero
    }
  }
  # Add up individual chi2 values for overall fit statistic
  fit.actual <- sum(chi2.actual[1:M, 1:J]) # Fit statistic for actual data set
  fit.sim <- sum(chi2.sim[1:M, 1:J])      # Fit statistic for a fitting model
  bpv <- step(fit.sim-fit.actual)        # Bayesian p-value
  c.hat <- fit.actual/fit.sim             # c-hat estimate
}

```

```

Ntotal <- sum(N[1:M])      # Total population size at M sites combined
p.derived <- alpha/(alpha+beta) #derived detection probability
rho.derived <- 1/(alpha+beta+1) #derived correlation coefficient

})

Nst <- apply(DWC, 1, max)+1
PBBmodel = nimbleModel(PBB_mixture, constants = win.data,
  inits = list(N = Nst, lambda = runif(1,0,5), alpha = runif(1,0.5,1.5),
    beta = runif(1,0.5,1.5)))
cPBBmodel <- compileNimble(PBBmodel)

PBBconf = configureMCMC(PBBmodel)
PBBconf$monitors
PBBconf$addMonitors(c("Ntotal", "fit.actual", "fit.sim", "c.hat",
  "bpv", "p.derived", "rho.derived", "lambda", "N", "alpha", "beta", "p"))

PBBmcmc = buildMCMC(PBBconf, enableWAIC = TRUE)

PBBmod = compileNimble(PBBmcmc, project = PBBmodel, resetFunctions = TRUE)
PBBmcmc.out = runMCMC(PBBmod, niter = 173000, nburnin = 4000,
  thin = 40, samplesAsCodaMCMC = TRUE, nchains = 2, WAIC = TRUE, summary = TRUE)
PBBmcmc.out$WAIC
PBBsummary = PBBmcmc.out$summary$all.chains[c('Ntotal', 'fit.actual',
  'fit.sim', 'c.hat', 'bpv', 'lambda', 'p.derived', 'rho.derived', 'alpha', 'beta'),]
PBBsummary

```

DW-BB N-mixture model

```

library(nimble)
dlmydweibull <- nimbleFunction(
  run = function(x = double(0), q = double(0), b = double(0),
    log = integer(0, default= 0)){
    returnType(double(0))
    logProb = log(q^(x^b) - q^((x+1)^b))
    if(log){
      return(logProb)
    }
    else{
      return(exp(logProb))
    }
  }
)

r1mydweibull <- nimbleFunction(
  run = function(n = double(0) ,q = double(0), b = double(0)){
    returnType(double(0))
    if(n != 1) print("r1mydweibull only allows n = 1; using n = 1.")
    u <- runif(1, min = 0, max = 1)
  }
)

```



```

    s <- ceiling((log(1 - u)/log(q))^(1/b)) - 1
    return(s)
  }
)

registerDistributions(list(
  d1mydweibull = list(
    BUGSdist = "d1mydweibull(q,b)",
    discrete = TRUE,
    range = c(0, Inf),
    types = c('value = integer()', 'q = double()', 'b = double()')
  )))

win.data = list(DWC = DWC, M= nrow(DWC), J = ncol(DWC), e = 0.001)
str(win.data)

N_mixdwbb <- nimbleCode( {

  # Priors

  #alpha ~dgamma(0.01,0.01)
  #beta ~dgamma(.01,.01)
  alpha ~ dexp(1)
  beta ~ dexp(1)
  q ~ dunif(0,1) # 0<q<1
  #b ~ dgamma(0.8,0.08)
  b ~ dexp(1) # b>0

  # Likelihood
  # Ecological model for true abundance
  for (i in 1:M) {
    N[i] ~ d1mydweibull(q,b)    # State model

    # Observation model for replicated counts
    for (j in 1:J) {
      DWC[i,j] ~ dbin(p[i,j], N[i])
      # Observation model, #counts follow a beta-binomial distribution
      p[i,j] ~ dbeta(alpha, beta)
      #detection probability p follows a beta distribution
    }
  }

  for (i in 1:27) {
    for (j in 1:4) {
      C.sim[i,j] ~ dbin(p[i,j], N[i]) # Create new data set under model
      e.count[i,j] <- N[i] * p[i,j]    # Expected datum
      # Chi-square discrepancy for the actual data
      chi2.actual[i,j] <- pow((DWC[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Chi-square discrepancy for the simulated ('perfect') data
      chi2.sim[i,j] <- pow((C.sim[i,j]-e.count[i,j]),2) / (e.count[i,j]+e)
      # Add small value e to denominator to avoid division by zero
    }
  }
}

```

```

    }
  }
  # Add up individual chi2 values for overall fit statistic
  fit.actual <- sum(chi2.actual[1:27, 1:4]) # Fit statistic for actual data set
  fit.sim <- sum(chi2.sim[1:27, 1:4])      # Fit statistic for a fitting model
  bpv <- step(fit.sim-fit.actual)        # Bayesian p-value
  c.hat <- fit.actual/fit.sim             # c-hat estimate

  # Derived quantities; unnecessary when running for inference purpose
  Ntotal <- sum(N[1:M])                  # Total population size at M sites combined
  p.derived <- alpha/(alpha+beta) #derived detection probability
  rho.derived <- 1/(alpha+beta+1)        #derived correlation coefficient

})

Nst <- apply(DWC, 1, max)+1              # Avoid data/model/inputs conflict

DWBBmodel = nimbleModel(N_mixdwbb, constants = win.data,
  inits = list(N = Nst,q = runif(1,0,1) ,b=rexp(1,1),
    alpha=runif(1,0.5,1.5), beta=runif(1,0.5,1.5)))
cDWBBmodel <- compileNimble(DWBBmodel)

DWBBconf = configureMCMC(DWBBmodel)
DWBBconf$monitors
DWBBconf$addMonitors(c("Ntotal", "fit.actual", "fit.sim", "c.hat",
  "bpv", "q", "b","p.derived", "rho.derived","alpha","beta","N","p" ))

DWBBmcmc = buildMCMC(DWBBconf, enableWAIC= TRUE)
DWBB = compileNimble(DWBBmcmc, project = DWBBmodel, resetFunctions = TRUE)
DWBBmcmc.out = runMCMC(DWBB, niter = 120000, nburnin = 4000,
  thin = 40, samplesAsCodaMCMC = TRUE, nchains = 2, WAIC = TRUE, summary = TRUE)
DWBBmcmc.out$WAIC
summary = DWBBmcmc.out$summary$all.chains[c('Ntotal', 'fit.actual',
  'fit.sim', 'c.hat', 'bpv', 'q', 'b' , 'p.derived','rho.derived','alpha', 'beta'),]
summary

```